

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №3
дисциплины «Искусственный интеллект и машинное обучение»
Вариант 1

Выполнил:

Аверин Максим Сергеевич

2 курс, группа ИВТ-б-о-24-1, 09.03.01

«Информатика и вычислительная
техника», очная форма обучения

(подпись)

Проверил:

Воронкин Р.А., доцент департамента
цифровых, робототехнических систем
и электроники института
перспективной инженерии

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2026 г.

Тема: основы работы с библиотекой matplotlib.

Цель: исследовать базовые возможности библиотеки matplotlib языка программирования Python.

Исходный репозиторий: https://github.com/ArXiTeKtOrS/Lab_3.git

Практическая часть:

Примеры.

1. Построение базового графика линейной функции $y=x$ на интервале $[0,10]$ с использованием 50 равномерно распределенных точек.

```
x = np.linspace(0,10,50) y
```

```
= x
```

```
plt.title(r'Линейная зависимость  $x=y$ ')
```

```
plt.xlabel(r' $x$ ', fontsize=12)
```

```
plt.ylabel(r' $y$ ', fontsize=12)
```

```
plt.grid(True) plt.plot(x,y) plt.show()
```

Результат выполнения:

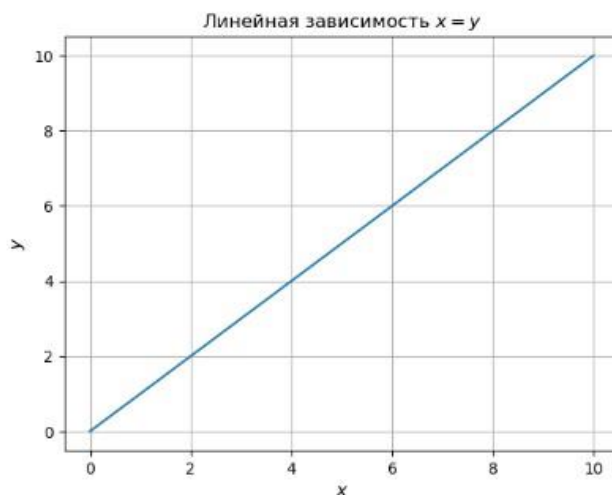


Рисунок 1 - Результат выполнения программы

2. Визуализация функции $y=\cos(\pi x)$. Демонстрирует настройку маркеров (`marker='o'`), цвета линии (`c='g'`), типа сетки (`linestyle="--"`) и прозрачности (`alpha=0.7`).

```
x = np.arange(0.0, 5, 0.01) y =  
np.cos(x * np.pi) plt.plot(x, y,  
marker='o', c='g')  
plt.title(r"График функции  $y = \cos(\pi x)$  (зеленые маркеры)",  
fontsize=14) plt.xlabel(r"Значение  $x$ ", fontsize=12)  
plt.ylabel(r"Значение  $y = \cos(\pi x)$ ", fontsize=12)  
plt.grid(True, linestyle="--", alpha=0.7) plt.show()
```

Результат выполнения:

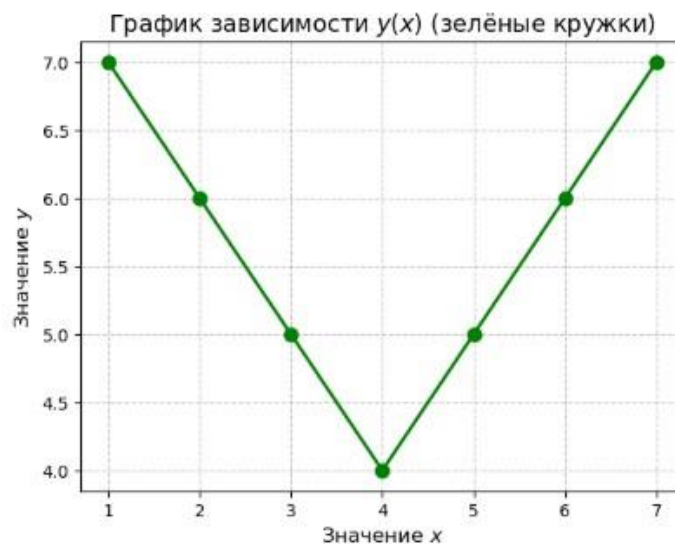


Рисунок 2 - Результат выполнения программы

3. Компактный формат строки форматирования "`r--`", где `r` задает красный цвет, а `--` задает пунктирный тип линии.

Показывает альтернативный синтаксис настройки стилей.

```
import numpy as np x =  
np.linspace(0,10,50) y =  
x  
plt.plot(x,y,"r--")
```

```
plt.title(r'Линейная зависимость $x=y$')  
plt.xlabel(r'$x$', fontsize=12)  
plt.ylabel(r'$y$', fontsize=12)  
plt.grid(True) plt.show()
```

Результат выполнения:

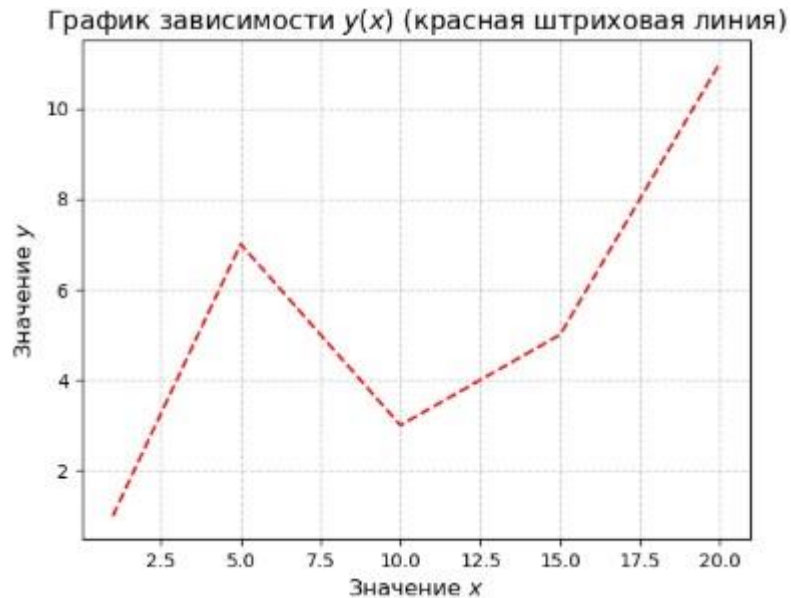


Рисунок 3 - Результат выполнения программы

4. Работа с объектом `plt.figure()` и передачей данных в виде стандартных Python-списков без использования `numpy`. Показывает базовый синтаксис библиотеки.

```
plt.figure() plt.plot([1,2,3,4,5],[1,2,3,4,5])  
plt.title(r"Линейная зависимость  
$y=x$") plt.xlabel(r'$x$', fontsize=12)  
plt.ylabel(r'$y$', fontsize=12)  
plt.grid(True) plt.show()
```

Результат выполнения:

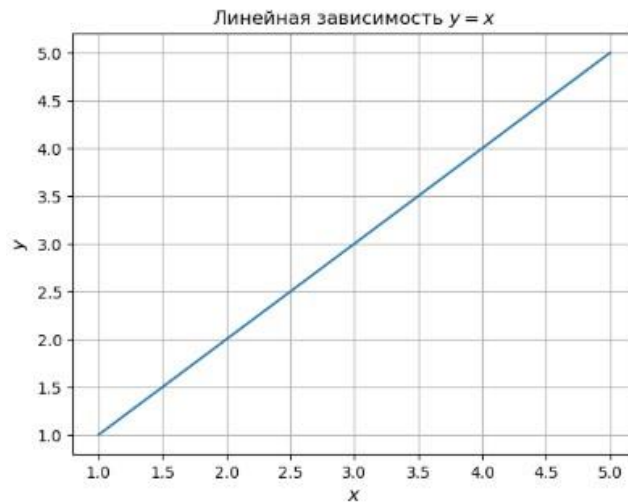


Рисунок 4 - Результат выполнения программы

5. Синтаксис `plt.plot([y1, y2, ...])`, где библиотека автоматически генерирует ось абсцисс как индексы массива `[0, 1, 2, ...]`. Показывает удобный способ быстрой визуализации списков значений.

```
plt.plot([1, 7, 3, 5, 11, 1])
```

```
plt.title(r"График функции  $y = f(x)$ ", fontsize=14)
```

```
plt.xlabel(r"Номер точки  $x$ ", fontsize=12)
```

```
plt.ylabel(r"Значение  $y$ ", fontsize=12)
```

```
plt.grid(True, linestyle="--", alpha=0.7) plt.show()
```

Результат выполнения:

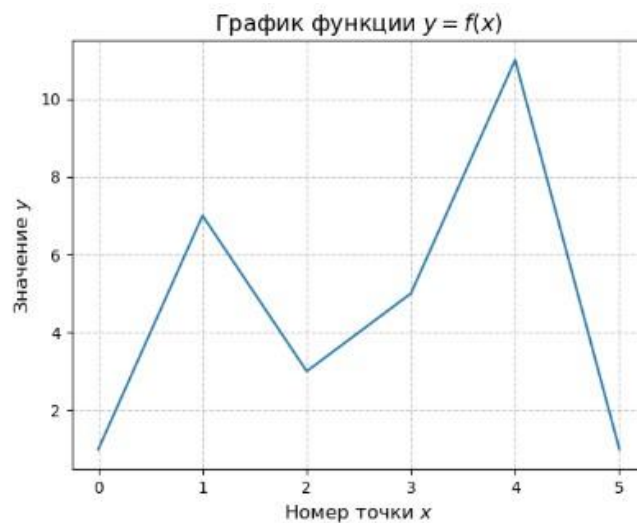


Рисунок 5 - Результат выполнения программы

6. Генерация 50 точек на отрезке $[0,10]$, построение графиков линейной и квадратичной зависимостей с оформлением заголовка, подписей осей и сетки. Функция `np.linspace(0, 10, 50)` генерирует 50 точек от 0 до 10 для массива x , $y_1 = x$ задаёт линейную зависимость, генератор списка `[i**2 for i in x]` – квадратичную, `plt.plot()` строит оба графика в одной системе координат, `plt.title()`, `plt.xlabel()`, `plt.ylabel()` оформляют подписи с поддержкой

LaTeX, `plt.grid(True)` включает сетку, а `plt.show()` выводит результат на экран.

```
x = np.linspace(0,10,50)
y1 = x
y2 = [i**2 for i
in x]

plt.title(r'Зависимости  $y_1 = x$  и  $y_2 = x^2$ ')
plt.xlabel(r' $x$ ', fontsize=12)
plt.ylabel(r' $y_1, y_2$ ', fontsize=12)
plt.grid(True)
plt.plot(x,y1,x,y2)
plt.show()
```

Результат выполнения:

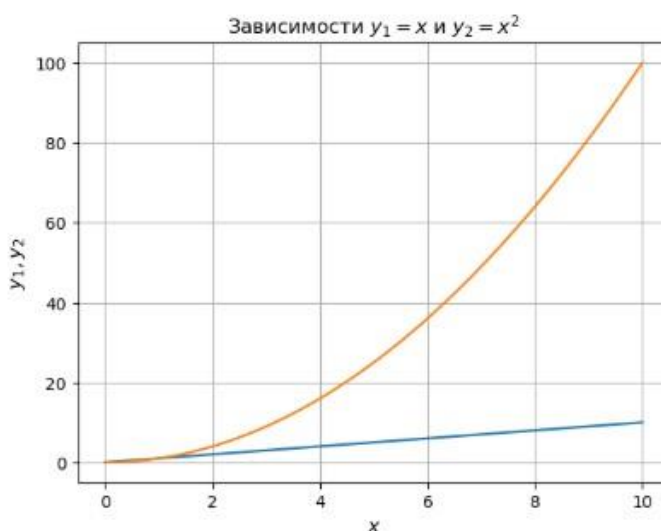


Рисунок 6 - Результат выполнения программы

7. Построение двух графиков (линейной и квадратичной зависимостей) на одном полотне с разделением на два подокна (`subplot`). Функция `np.linspace(0,10,50)` создаёт массив x из 50 точек. $y_1 = x$ задаёт линейную

зависимость, генератор $[i**2 \text{ for } i \text{ in } x]$ – квадратичную. `plt.figure(figsize=(9,9))` задаёт размер полотна. `plt.subplot(2,1,1)` активирует верхнюю область для графика y_1 . `plt.subplot(2,1,2)` активирует нижнюю область для графика y_2 . `plt.show()` выводит результат.

```
x = np.linspace(0,10,50) y1 = x y2  
= [i**2 for i in x]  
plt.figure(figsize=(9,9))  
plt.subplot(2,1,1) plt.plot(x,y1)  
plt.title(r'Зависимость $y_1 =  
x$') plt.ylabel(r'$y_1$',  
fontsize=14) plt.grid(True)
```

```
plt.subplot(2,1,2) plt.plot(x, y2)  
plt.xlabel(r'$x$', fontsize=14)  
plt.ylabel(r'$y_2$', fontsize=14)  
plt.grid(True) plt.show()
```

Результат выполнения:

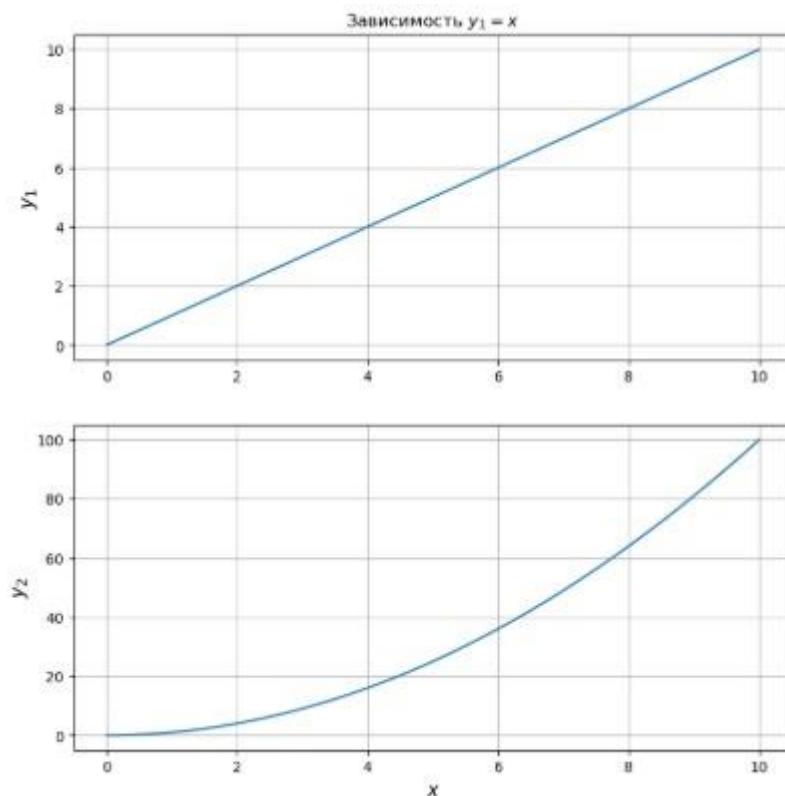


Рисунок 7 - Результат выполнения программы

8. Функция `np.linspace(0,10,10)` создаёт массив `x` из 10 точек. `y1 = 4*x` задаёт линейную зависимость, генератор `[i**2 for i in x]` – квадратичную. `fig, ax = plt.subplots()` создаёт фигуру и оси. `ax.set_title()`, `set_xlabel()`, `set_ylabel()` оформляют заголовок и подписи осей. `ax.grid(which="major")` настраивает основную сетку, `ax.grid(which="minor")` – вспомогательную (пунктирную). `ax.scatter(x, y2)` строит точечный график для `y2`, `ax.legend()` отображает легенду. `AutoMinorLocator()` добавляет метки вспомогательных делений. `ax.tick_params()` настраивает длину и толщину основных и вспомогательных рисок. `plt.show()` выводит результат.

```
from matplotlib.ticker import (MultipleLocator, FormatStrFormatter,
AutoMinorLocator) import numpy
as np import matplotlib.pyplot
as plt
```

```
x = np.linspace(0,10,10)
y1 = 4*x y2 = [i**2
for i in x]
```

```
fig, ax = plt.subplots(figsize = (8,6)) ax.set_title(r"$Y_1$
= 4X, \ $Y_2 = X^2$", fontsize=16) ax.set_xlabel(r"$x$",
fontsize=14) ax.set_ylabel(r"$y_1, y_2$", fontsize=14)
```

```
ax.grid(which="major", linewidth=1.2) ax.grid(which="minor",
linestyle="--", color="gray", linewidth=0.5)
```

```
ax.scatter(x, y2, label=r"$y_2 = x^2$") ax.legend()
```



```
ax.xaxis.set_minor_locator(AutoMinorLocator())
```

```
ax.yaxis.set_minor_locator(AutoMinorLocator())
```

```
ax.tick_params(which='major', length=10, width=2)
```

```
ax.tick_params(which='minor', length=5, width=1)
```

```
plt.show()
```

Результат выполнения:

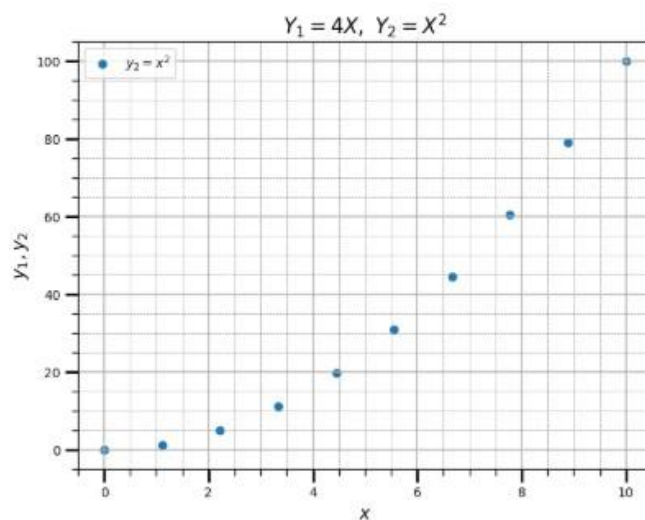


Рисунок 8 - Результат выполнения программы

9. Списки x и y задают координаты точек (день и цена). `plt.plot()` строит линейный график с подписью легенды. `plt.title()`, `plt.xlabel()`, `plt.ylabel()` оформляют заголовок и подписи осей (синий цвет для осей). `plt.legend()` отображает легенду. `plt.grid(True, linestyle="--", alpha=0.7)` включает пунктирную сетку с прозрачностью 70%. `plt.text(15, 4)` добавляет текстовую метку "grow up!" в координатах (15,4). `plt.show()` выводит результат.

```
x = [1, 5, 10, 15, 20] y = [1, 7, 3, 5, 11] plt.plot(x, y,  
label=r"$\text{steel price}$") plt.title(r"$\text{Chart  
price}$", fontsize=15) plt.xlabel(r"$\text{Day}$",
```

```
fontsize=12, color='blue') plt.ylabel(r"$\text{Price}$",  
fontsize=12, color='blue')
```

```
plt.legend() plt.grid(True, linestyle="--  
", alpha=0.7) plt.text(15, 4,  
r"$\text{grow up!}$") plt.show()
```

Результат выполнения:

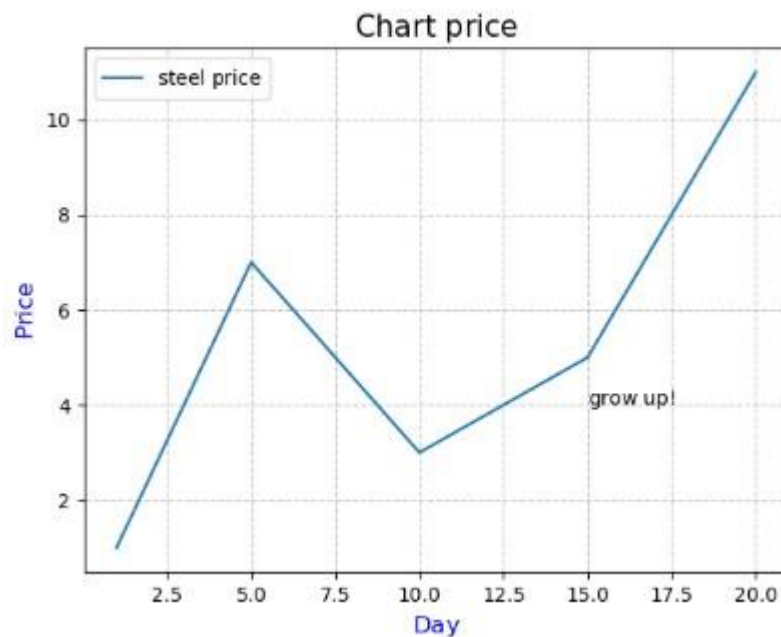


Рисунок 9 - Результат выполнения программы

10. Списки x и y задают координаты точек. `plt.plot(x, y, 'ro')` строит точечный график, где 'r' задаёт красный цвет, а 'o' – форму маркера (кружок). `plt.title()` оформляет заголовок, `plt.xlabel()` и `plt.ylabel()` подписывают оси. `plt.grid(True, linestyle="--", alpha=0.7)` включает пунктирную сетку с прозрачностью. `plt.show()` выводит результат.

```
x = [1, 5, 10, 15, 20] y  
= [1, 7, 3, 5, 11]
```

```
plt.plot(x, y, 'ro')
```

```
plt.title(r"Точечный график зависимости $y(x)$ (красные кружки)",
fontsize=14) plt.xlabel(r"Значение $x$",
fontsize=12) plt.ylabel(r"Значение $y$",
fontsize=12) plt.grid(True, linestyle="--", alpha=0.7) plt.show()
```

Результат выполнения:

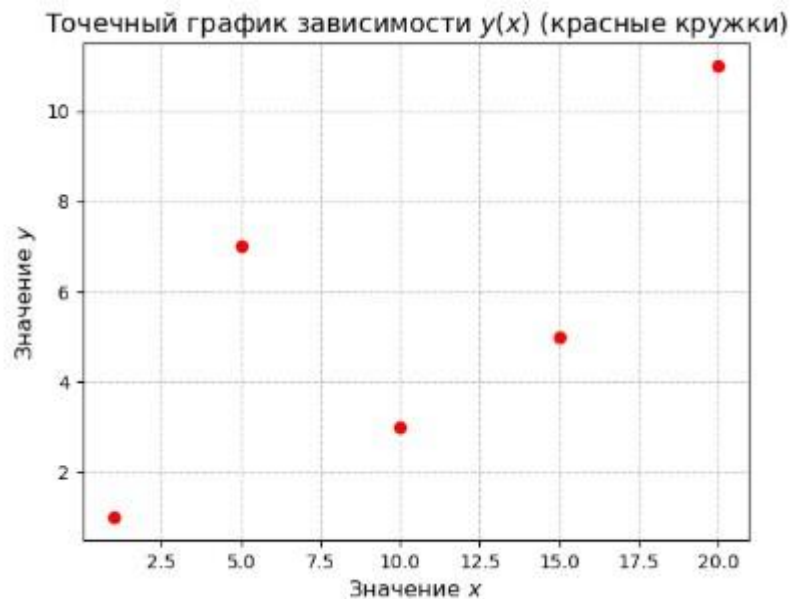


Рисунок 10 - Результат выполнения программы

11. Список x задаёт координаты по горизонтали, $y1$ – исходные значения. Генераторы списков $[i*1.2 + 1 \text{ for } i \text{ in } y1]$ последовательно создают $y2$, $y3$, $y4$ путём масштабирования и сдвига предыдущего массива. `plt.figure(figsize=(12,7))` задаёт размер полотна. Четыре вызова `plt.subplot(2,2,1)...`(2,2,4) разбивают полотно на 2×2 и строят графики в каждом подокне с разными типами линий: сплошная ('-'), пунктирная ('--'), штрих-пунктирная ('-.'), и точечная (':'). `plt.tight_layout()` автоматически настраивает отступы между подокнами. `plt.show()` выводит результат.

```
x = [1, 5, 10, 15, 20] y1 =
[1, 7, 3, 5, 11] y2 = [i*1.2
+ 1 for i in y1] y3 = [i*1.2
```

```
+ 1 for i in y2] y4 = [i*1.2  
+ 1 for i in y3]
```

```
plt.figure(figsize=(12,7))
```

```
plt.subplot(2,2,1) plt.plot(x,  
y1, '-') plt.title(r"$y_1(x)$",  
fontsize=12)  
plt.xlabel(r"$x$",  
fontsize=10)  
plt.ylabel(r"$y_1$",  
fontsize=10) plt.grid(True,  
linestyle="--", alpha=0.7)
```

```
plt.subplot(2,2,2) plt.plot(x, y2, '--')  
plt.title(r"$y_2(x)$", fontsize=12)  
plt.xlabel(r"$x$", fontsize=10)  
plt.ylabel(r"$y_2$", fontsize=10)  
plt.grid(True, linestyle="--", alpha=0.7)
```

```
plt.subplot(2,2,3) plt.plot(x, y3, '-.')  
plt.title(r"$y_3(x)$", fontsize=12)  
plt.xlabel(r"$x$", fontsize=10)  
plt.ylabel(r"$y_3$", fontsize=10)  
plt.grid(True, linestyle="--", alpha=0.7)
```

```
plt.subplot(2,2,4) plt.plot(x, y4, ':')  
plt.title(r"$y_4(x)$", fontsize=12)  
plt.xlabel(r"$x$", fontsize=10)
```

```
plt.ylabel(r"$y_4$", fontsize=10)
plt.grid(True, linestyle="--", alpha=0.7)
```

```
plt.tight_layout() plt.show()
```

Результат выполнения:

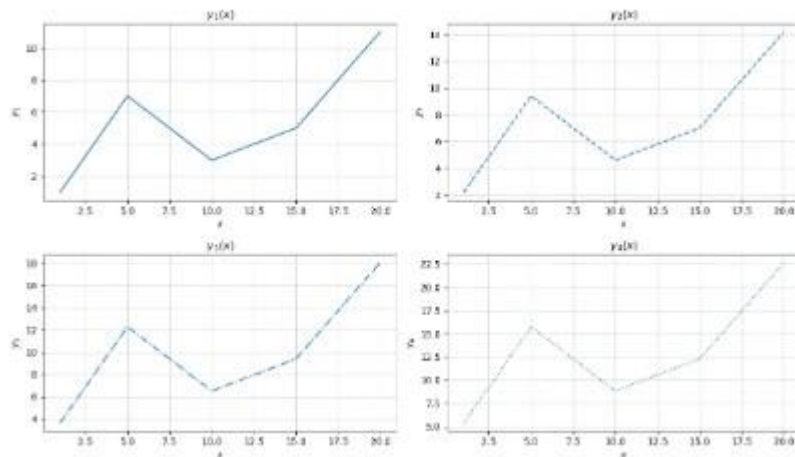


Рисунок 11 - Результат выполнения программы

12. Пример демонстрирует тот же самый принцип, что и предыдущий, но показана возможность не писать через запятую параметры `plt.subplot(224)`.

```
x = [1, 5, 10, 15, 20] y1 =
[1, 7, 3, 5, 11] y2 = [i*1.2
+ 1 for i in y1] y3 = [i*1.2
+ 1 for i in y2] y4 = [i*1.2
+ 1 for i in y3]
```

```
plt.subplot(221) plt.plot(x, y1, '-')
plt.title(r"$y_1(x)$", fontsize=12)
plt.xlabel(r"$x$", fontsize=10)
plt.ylabel(r"$y_1$", fontsize=10)
plt.grid(True, linestyle="--", alpha=0.7)
```

```
plt.subplot(222) plt.plot(x, y2, '--  
) plt.title(r"$y_2(x)$",  
fontsize=12) plt.xlabel(r"$x$",  
fontsize=10) plt.ylabel(r"$y_2$",  
fontsize=10) plt.grid(True,  
linestyle="--", alpha=0.7)
```

```
plt.subplot(223) plt.plot(x, y3, '-.')  
plt.title(r"$y_3(x)$", fontsize=12)  
plt.xlabel(r"$x$", fontsize=10)  
plt.ylabel(r"$y_3$", fontsize=10)  
plt.grid(True, linestyle="--", alpha=0.7)
```

```
plt.subplot(224) plt.plot(x, y4, ':')  
plt.title(r"$y_4(x)$", fontsize=12)  
plt.xlabel(r"$x$", fontsize=10)  
plt.ylabel(r"$y_4$", fontsize=10)  
plt.grid(True, linestyle="--", alpha=0.7)
```

```
plt.tight_layout() plt.show()
```

Результат выполнения:

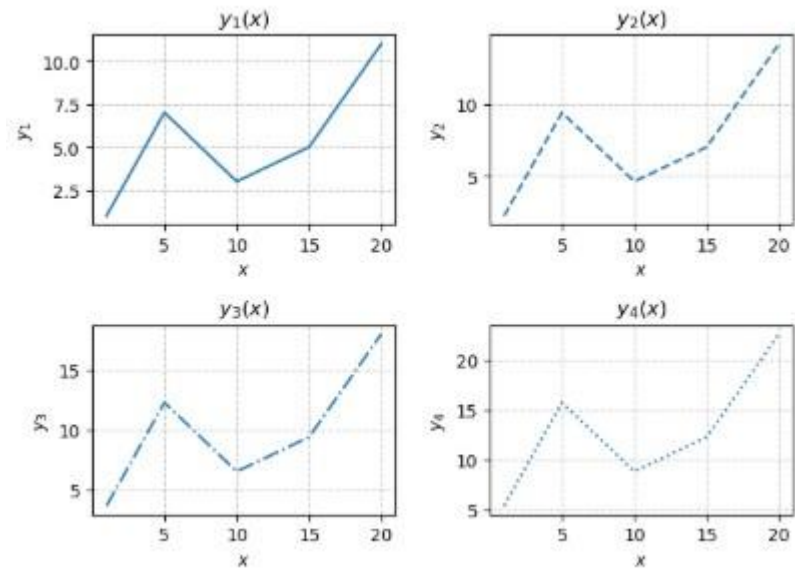


Рисунок 12 - Результат выполнения программы

13. `fig, axs = plt.subplots(2, 2, figsize=(12,7))` создаёт фигуру и сетку осей 2×2 . Обращение к подокнам осуществляется через индексы `axs[0,0]`, `axs[0,1]`, `axs[1,0]`, `axs[1,1]`. Для каждого подокна методом `.plot()` строится график с разными типами линий: сплошная ('-'), пунктирная ('--'), штрих-пунктирная ('-.'), и точечная (':'). Методы `.set_title()`, `.set_xlabel()`, `.set_ylabel()` и `.grid()` настраивают заголовки, подписи осей и сетку. `plt.tight_layout()` корректирует отступы, `plt.show()` выводит результат.

```
x = [1, 5, 10, 15, 20] y1 =
[1, 7, 3, 5, 11] y2 = [i*1.2
+ 1 for i in y1] y3 = [i*1.2
+ 1 for i in y2] y4 = [i*1.2
+ 1 for i in y3]
```

```
fig, axs = plt.subplots(2, 2, figsize=(12, 7))
```

```
axs[0, 0].plot(x, y1, '-') axs[0,
0].set_title(r"$y_1(x)$", fontsize=12) axs[0,
0].set_xlabel(r"$x$", fontsize=10) axs[0,
```

```
0].set_ylabel(r"$y_1$", fontsize=10) axs[0,  
0].grid(True, linestyle="--", alpha=0.7)
```

```
axs[0, 1].plot(x, y2, '--') axs[0,  
1].set_title(r"$y_2(x)$", fontsize=12) axs[0,  
1].set_xlabel(r"$x$", fontsize=10) axs[0,  
1].set_ylabel(r"$y_2$", fontsize=10) axs[0,  
1].grid(True, linestyle="--", alpha=0.7)
```

```
axs[1, 0].plot(x, y3, '-.') axs[1,  
0].set_title(r"$y_3(x)$", fontsize=12) axs[1,  
0].set_xlabel(r"$x$", fontsize=10) axs[1,  
0].set_ylabel(r"$y_3$", fontsize=10) axs[1,  
0].grid(True, linestyle="--", alpha=0.7)
```

```
axs[1, 1].plot(x, y4, ':') axs[1,  
1].set_title(r"$y_4(x)$", fontsize=12) axs[1,  
1].set_xlabel(r"$x$", fontsize=10) axs[1,  
1].set_ylabel(r"$y_4$", fontsize=10) axs[1,  
1].grid(True, linestyle="--", alpha=0.7)
```

```
plt.tight_layout() plt.show()
```

Результат выполнения:

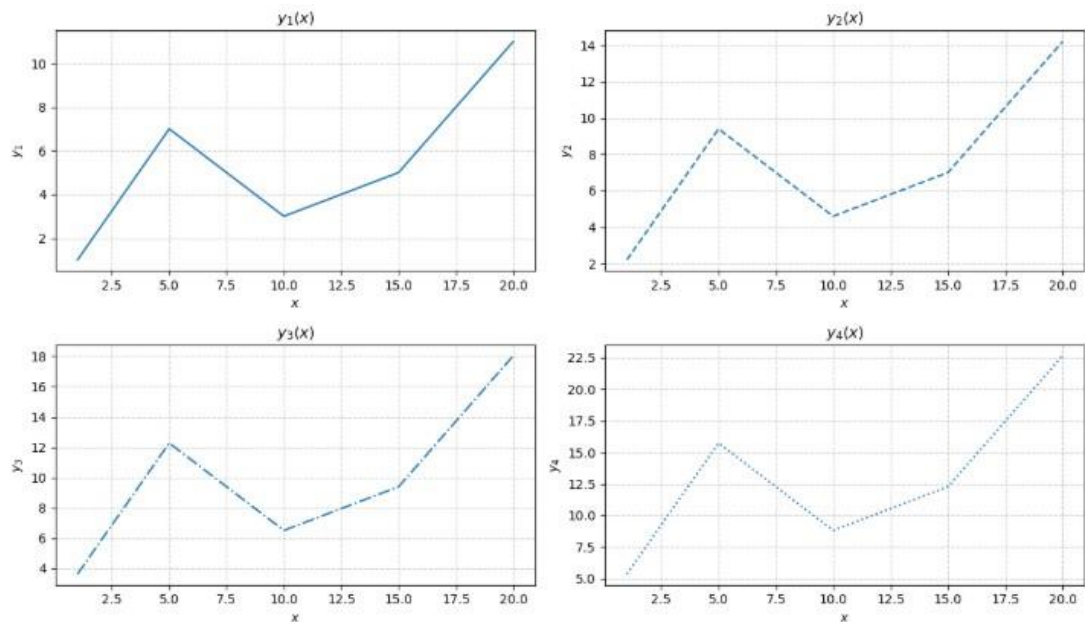


Рисунок 13 - Результат выполнения программы

14. Списки x , y_1 , y_2 задают координаты двух графиков. `plt.plot(x, y1, 'o-r', alpha=0.7, label=r"$\text{first}$", lw=5, mec='b', mew=2, ms=10)` строит первый график: 'o-r' означает круглые маркеры (o), сплошную линию (-) и красный цвет (r); `alpha` – прозрачность, `lw` – толщина линии, `mec` – цвет обводки маркера, `mew` – толщина обводки, `ms` – размер маркера. Второй график `plt.plot(x, y2, 'v-.g', ...)` использует треугольные маркеры (v), штрихпунктирную линию (-.) и зелёный цвет (g). `plt.legend()` отображает легенду с подписями "first" и "second". `plt.show()` выводит результат.

```
x = [1, 5, 10, 15, 20]
y1 = [1, 7, 3, 5, 11]
y2 = [4, 3, 1, 8, 12]
```

```
plt.figure(figsize=(12, 7)) plt.plot(x, y1, 'o-r', alpha=0.7,
label=r"$\text{first}$", lw=5, mec='b',
mew=2, ms=10) plt.plot(x, y2, 'v-.g', label=r"$\text{second}$", mec='r', lw=2,
mew=2,
ms=12)
```

```
plt.title(r"Графики зависимостей  $y_1(x)$  и  $y_2(x)$ ", fontsize=16)
plt.xlabel(r"Значение  $x$ ", fontsize=14) plt.ylabel(r"Значение  $y$ ",
fontsize=14) plt.legend() plt.grid(True, linestyle="--", alpha=0.7)
plt.show()
```

Результат выполнения:

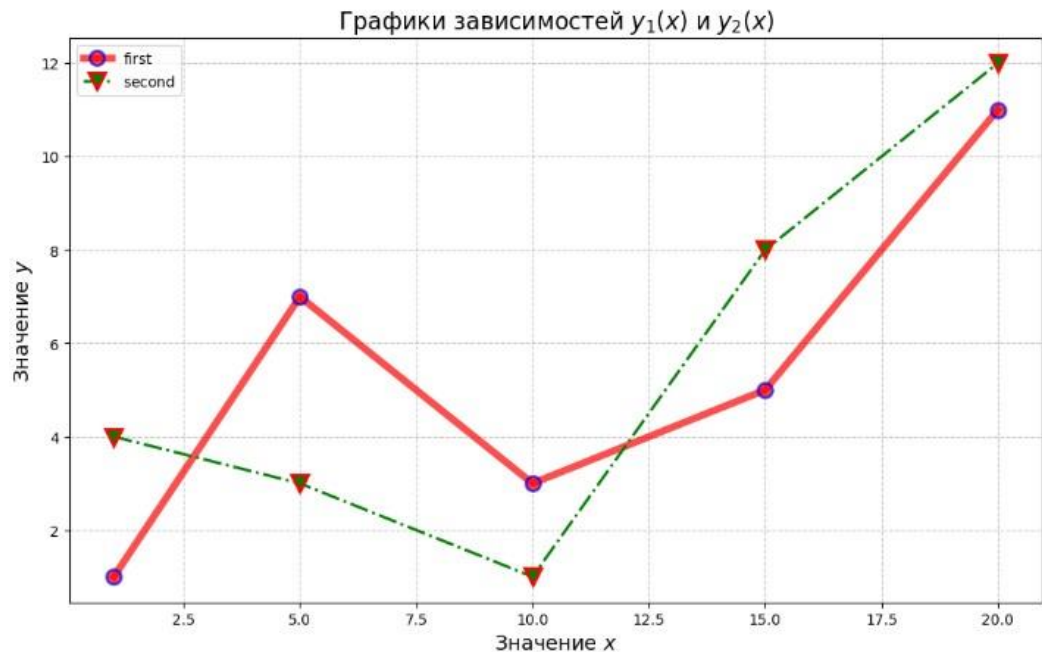


Рисунок 14 - Результат выполнения программы

15. `plt.plot(x, y, c='r')` строит график красного цвета. `plt.fill_between(x, y)` выполняет заливку области под кривой между осью X и графиком. `plt.title()`, `plt.xlabel()`, `plt.ylabel()` оформляют заголовок и подписи осей. `plt.grid()` включает пунктирную сетку. `plt.show()` выводит результат.

```
plt.plot(x, y, c='r') plt.fill_between(x,
y)
```

```
plt.title(r"График функции  $y(x)$  с заливкой под кривой", fontsize=14)
plt.xlabel(r"Значение  $x$ ", fontsize=12)
```

```
plt.ylabel(r"Значение $y$", fontsize=12)
plt.grid(True, linestyle="--", alpha=0.7) plt.show()
```

Результат выполнения:

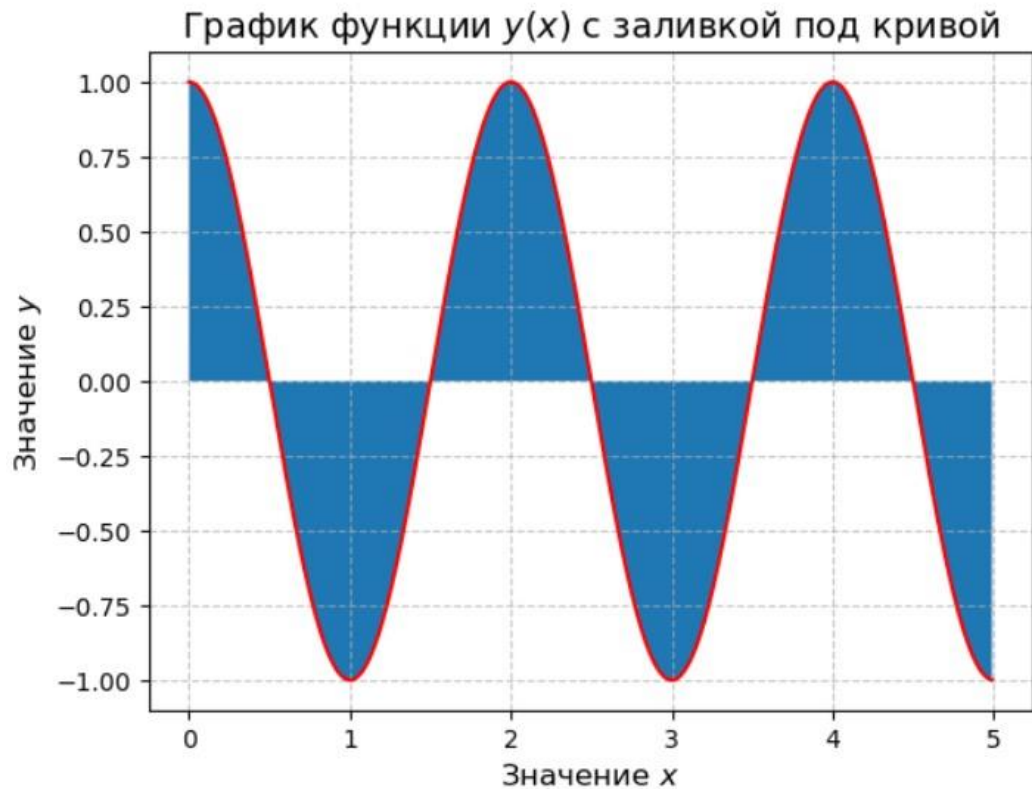


Рисунок 15 - Результат выполнения программы

16. `plt.plot(x, y, c='r')` строит график красного цвета. `plt.fill_between(x, y, where=(y > 0.75) | (y < -0.75))` выполняет заливку только тех областей под кривой, где значение y больше 0.75 или меньше -0.75 (то есть $|y| > 0.75$). Параметр `where` задаёт условие для выборочной заливки. `plt.show()` выводит результат.

```
plt.plot(x, y, c='r') plt.fill_between(x, y, where=(y >
0.75) | (y < -0.75))
```

```
plt.title(r"График функции $y(x)$ с заливкой областей $|y| > 0.75$",
fontsize=14)
```

```
plt.xlabel(r"Значение $x$", fontsize=12)
plt.ylabel(r"Значение $y$", fontsize=12)
plt.grid(True, linestyle="--", alpha=0.7) plt.show()
```

Результат выполнения:

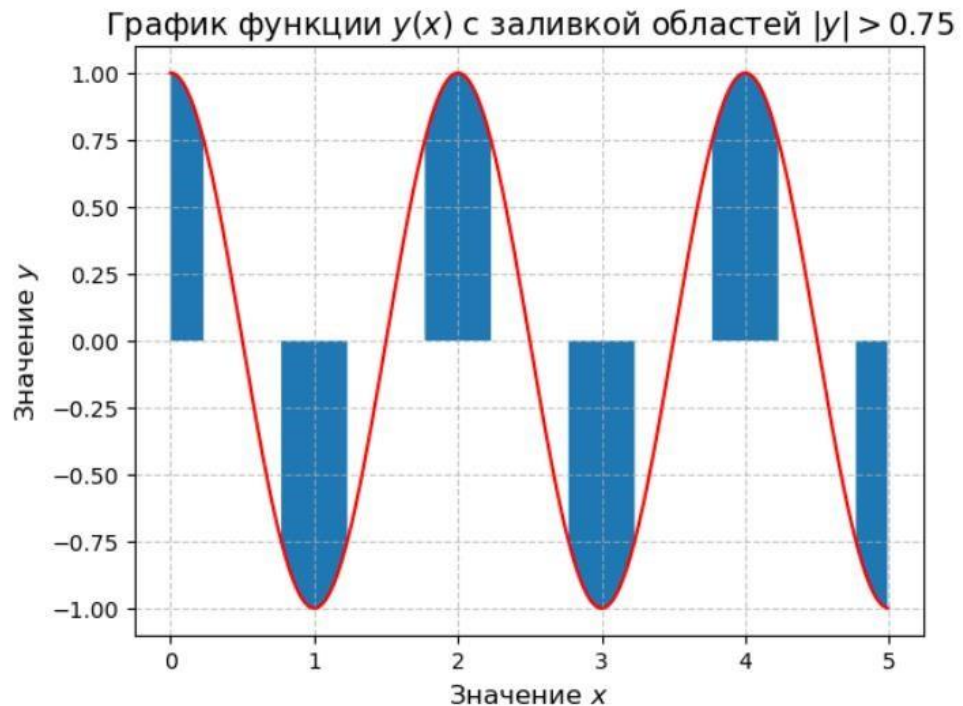


Рисунок 16 - Результат выполнения программы

17. `plt.plot(x, y, c='r')` строит график красного цвета. Первый вызов `plt.fill_between(x, y, where=(y >= 0), color='g', alpha=0.3)` заливает зелёным цветом с прозрачностью 30% области, где $y \geq 0$ (выше оси X). Второй вызов `plt.fill_between(x, y, where=(y <= 0), color='r', alpha=0.3)` заливает красным цветом с прозрачностью 30% области, где $y \leq 0$ (ниже оси X). `plt.grid()` включает пунктирную сетку. `plt.show()` выводит результат.

```
plt.plot(x, y, c='r') plt.grid(True, linestyle="--", alpha=0.7)
plt.fill_between(x, y, where=(y >= 0), color='g', alpha=0.3)
plt.fill_between(x, y, where=(y <= 0), color='r', alpha=0.3)
plt.title(r"График функции $y(x)$ с разноцветной
заливкой: $y \ge 0$ и
```

```

$y <= 0$", fontsize=14) plt.xlabel(r"Значение
    $x$", fontsize=12) plt.ylabel(r"Значение
    $y$", fontsize=12) plt.show()

```

Результат выполнения:

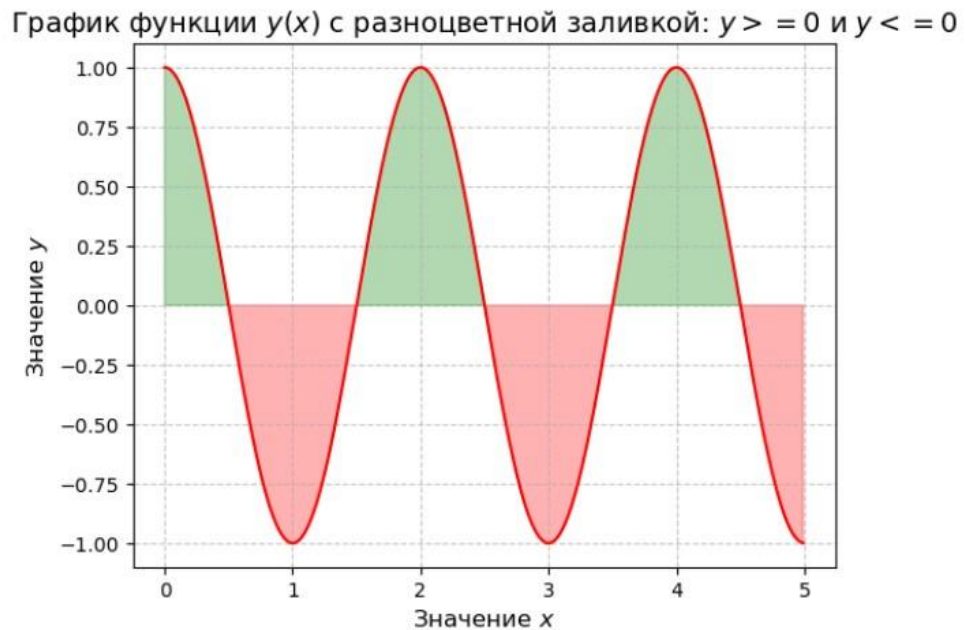


Рисунок 17 - Результат выполнения программы

18. Функции `np.arange(0, 5, 0.01)` и `np.cos(x * np.pi)` генерируют данные для графика косинуса. Список `m_ev_case` содержит различные способы задания параметра `markevery` (управляет частотой отображения маркеров): `None` (все маркеры), `10` (каждый 10-й), `(100, 30)` (начало с 100, затем каждый 30-й), `slice(100, 400, 15)` (срез), списки индексов. Цикл `for i, case in enumerate(m_ev_case)` перебирает варианты и строит графики в сетке `plt.subplots(2, 3)`. В каждом подокне `axs[i].plot(x, y, 'o', ls='-', ms=7, markevery=case)` строит график с круглыми маркерами и сплошной линией, где `markevery=case` задаёт выбранный вариант отображения маркеров. `plt.show()` выводит результат.

```

x = np.arange(0.0, 5, 0.01) y
= np.cos(x * np.pi)

```

```
m_ev_case = [None, 10, (100, 30), slice(100, 400, 15), [0, 100, 200, 300],
[10, 50, 100]]
```

```
fig, ax = plt.subplots(2, 3, figsize=(10, 7))
axs = [ax[i, j] for i in range(2) for j in range(3)]
```

```
for i, case in enumerate(m_ev_case):
```

```
    axs[i].set_title(rf"markevery = {case}", fontsize=12)
```

```
    axs[i].plot(x, y, 'o', ls='-', ms=7, markevery=case)
```

```
    axs[i].set_xlabel(r"$x$", fontsize=10)    axs[i].set_ylabel(r"$y$")
    axs[i].set_ylabel(r"$y = \cos(\pi x)$", fontsize=10)    axs[i].grid(True, linestyle="--",
    alpha=0.7)
```

```
plt.tight_layout() plt.show()
```

Результат выполнения:

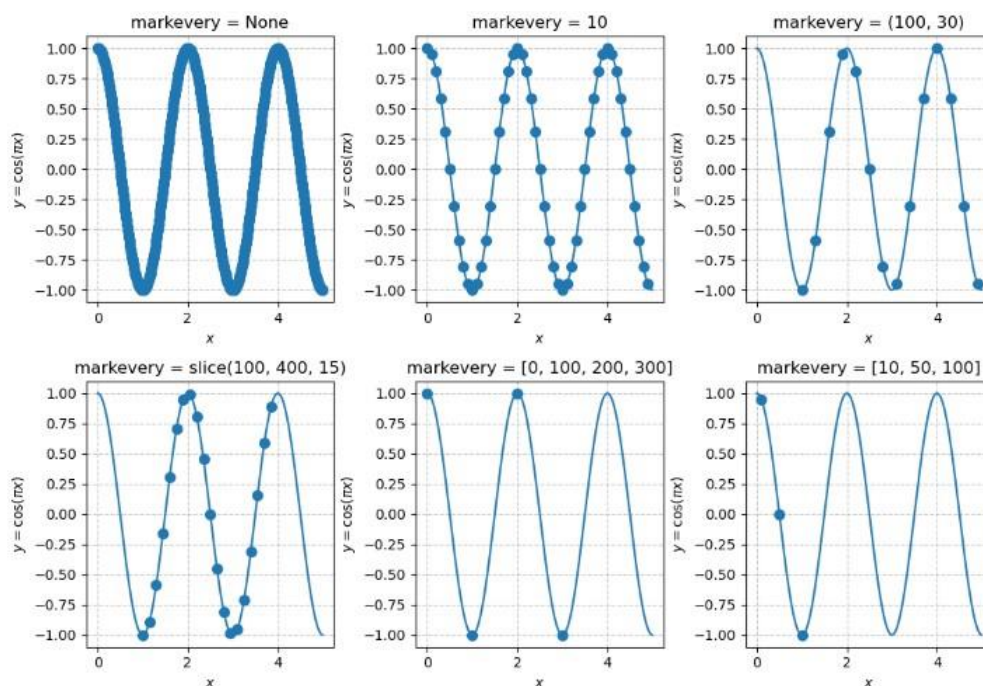


Рисунок 18 - Результат выполнения программы

19. `np.ma.masked_where(y < -0.5, y)` создаёт маскированный массив, в котором все значения `y`, меньшие `-0.5`, скрываются (не отображаются на графике). `plt.ylim(-1, 1)` задаёт пределы вертикальной оси от `-1` до `1`. `plt.plot(x, y_masked, linewidth=3, color='b')` строит график с толщиной линии `3`, синего цвета; участки, где `y < -0.5`, отсутствуют (разрывы). `plt.legend()` отображает легенду. `plt.show()` выводит результат.

```
x = np.arange(0.0, 5, 0.01) y = np.cos(x *  
np.pi) y_masked = np.ma.masked_where(y <  
-0.5, y) plt.ylim(-1, 1)
```

```
plt.plot(x, y_masked, linewidth=3, color='b', label=r"$y = \cos(\pi x)$")
```

```
plt.title(r"График функции $y = \cos(\pi x)$ с маскировкой $y < -0.5$",  
fontsize=14) plt.xlabel(r"Значение $x$",  
fontsize=12) plt.ylabel(r"Значение $y$",  
fontsize=12) plt.grid(True, linestyle="--  
", alpha=0.7) plt.legend() plt.show()
```

Результат выполнения:

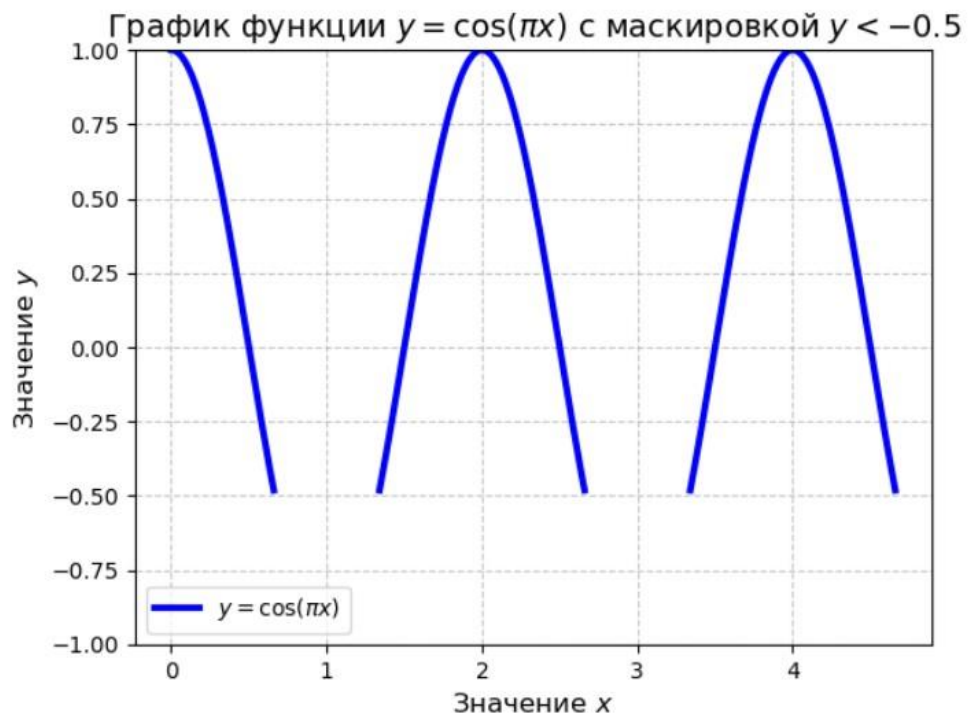


Рисунок 19 - Результат выполнения программы

20. `np.arange(0, 7)` и $y = x$ генерируют данные для линейной функции. Список `where_set = ['pre', 'post', 'mid']` содержит три варианта позиционирования ступенек. В цикле `for i, ax in enumerate(axes)` для каждого подокна вызывается `ax.step(x, y, 'g-o', where=where_set[i])`, где `step()` строит ступенчатый график: 'g-o' – зелёные маркеры-кружки со сплошной линией; параметр `where` задаёт расположение горизонтального сегмента ступеньки: 'pre' – значение отображается в начале интервала, 'post' – в конце, 'mid' – по середине. `plt.suptitle()` добавляет общий заголовок над всеми подокнами. `plt.show()` выводит результат.

```
x = np.arange(0, 7) y
= x
```

```
where_set = ['pre', 'post', 'mid'] fig, axes =
plt.subplots(1, 3, figsize=(15, 4))
```

```
for i, ax in enumerate(axes):
```



```

ax.step(x, y, 'g-o', where=where_set[i])    ax.set_title(rf"$\text{{where}}$")
=      \text{{'{{where_set[i]}}'}}$,
fontsize=14)    ax.set_xlabel(r"$x$",
               fontsize=12)    ax.set_ylabel(r"$y = x$",
               fontsize=12)    ax.grid(True, linestyle="-
               -", alpha=0.7)

```

```

plt.suptitle(r"Демонстрация параметра where в функции step() для
графика $y = x$", fontsize=16)
plt.tight_layout() plt.show()

```

Результат выполнения:

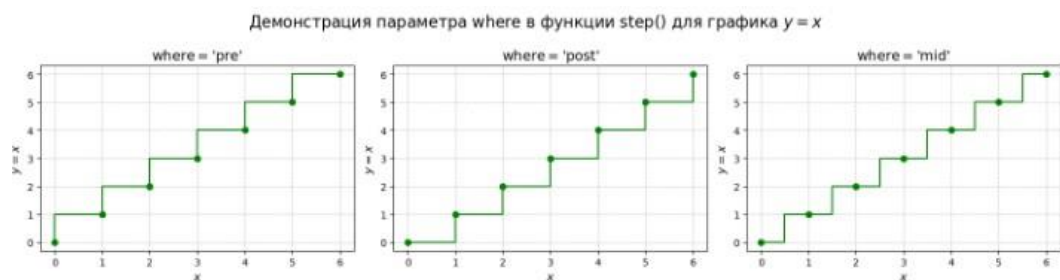


Рисунок 20 - Результат выполнения программы

21. Функции `np.arange(0, 11, 1)` генерирует массив `x` от 0 до 10. Генераторы списков создают массивы `y1`, `y2`, `y3` для квадратичных и линейной функций. `ax.stackplot(x, y1, y2, y3, labels=labels)` строит диаграмму с накоплением (stackplot), где области под графиками суммируются по вертикали. `ax.legend(loc='upper left')` размещает легенду в левом верхнем углу. `plt.show()` выводит результат.

```
x = np.arange(0, 11, 1)
```

```

y1 = np.array([(-0.2)*i**2 + 2*i for i in x]) y2
= np.array([(-0.4)*i**2 + 4*i for i in x]) y3 =
np.array([2*i for i in x])

```

```
labels = ["y1", "y2", "y3"]
```

```
fig, ax = plt.subplots()
```

```
ax.stackplot(x, y1, y2, y3, labels=labels)
```

```
ax.legend(loc='upper left')
```

```
plt.title(r'График для функций $y_1(x)$, $y_2(x)$ и $y_3(x)$')
```

```
plt.xlabel(r'Ось $x$') plt.ylabel(r'Ось $y$') plt.grid(True)
```

```
plt.show()
```

Результат выполнения:

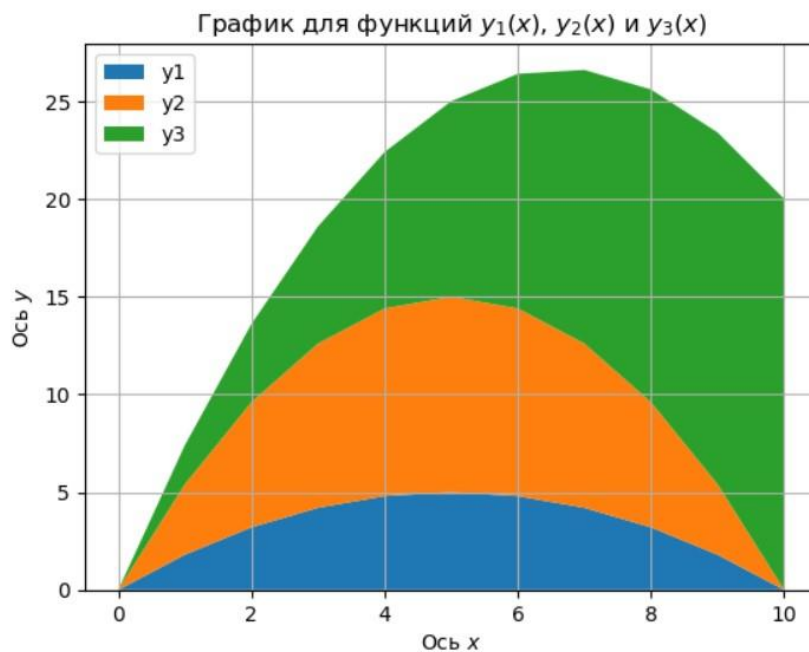


Рисунок 21 - Результат выполнения программы

22. `np.arange(0, 10.5, 0.5)` генерирует массив x с шагом 0.5. Генератор списка вычисляет $y = -0.2x^2 + 2x$. `plt.stem(x, y)` строит стеблевой график (вертикальные линии с маркерами на концах). Оформляются заголовок, подписи осей и сетка. `plt.show()` выводит результат.

```
x = np.arange(0, 10.5, 0.5) y =  
np.array([(-0.2)*i**2 + 2*i for i in x])
```

```
plt.stem(x, y)  
plt.title(r'Стеблевой график для функции $y(x) = -0.2x^2 + 2x$')  
plt.xlabel(r'Ось $x$') plt.ylabel(r'Ось $y$') plt.grid(True)  
plt.show()
```

Результат выполнения:

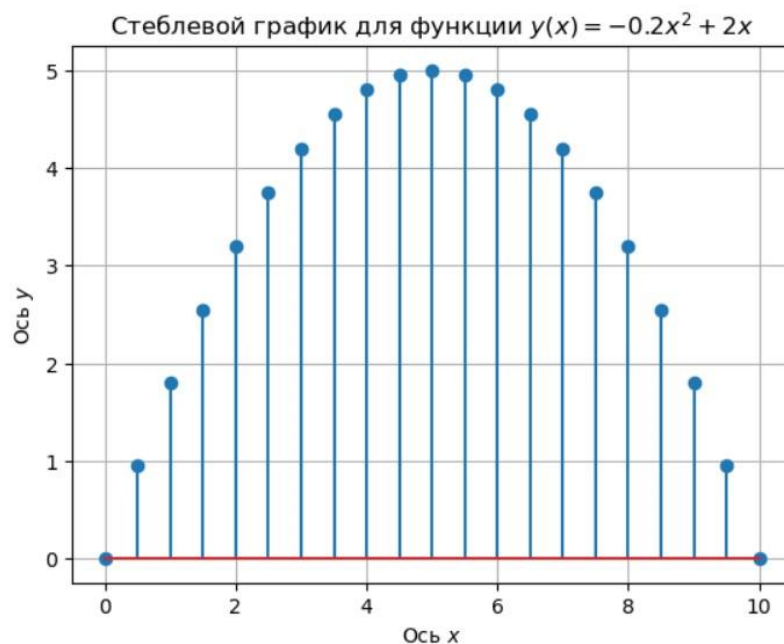


Рисунок 22 - Результат выполнения программы

23. `plt.stem(x, y, linefmt="b--", markerfmt="^", bottom=1)` строит стеблевой график с синей пунктирной линией (b--), маркерами в виде треугольников (^) и нижней границей `bottom=1`. Остальное оформление аналогично предыдущему примеру.

```
x = np.arange(0, 10.5, 0.5) y =  
np.array([(-0.2)*i**2 + 2*i for i in x])
```

```
plt.stem(x, y, linefmt="b--", markerfmt="^", bottom=1)
plt.title(r'Стеблевой график с настройками: линия --, маркер ^')
plt.xlabel(r'Ось $x$') plt.ylabel(r'Ось $y$') plt.grid(True)
plt.show()
```

Результат выполнения:

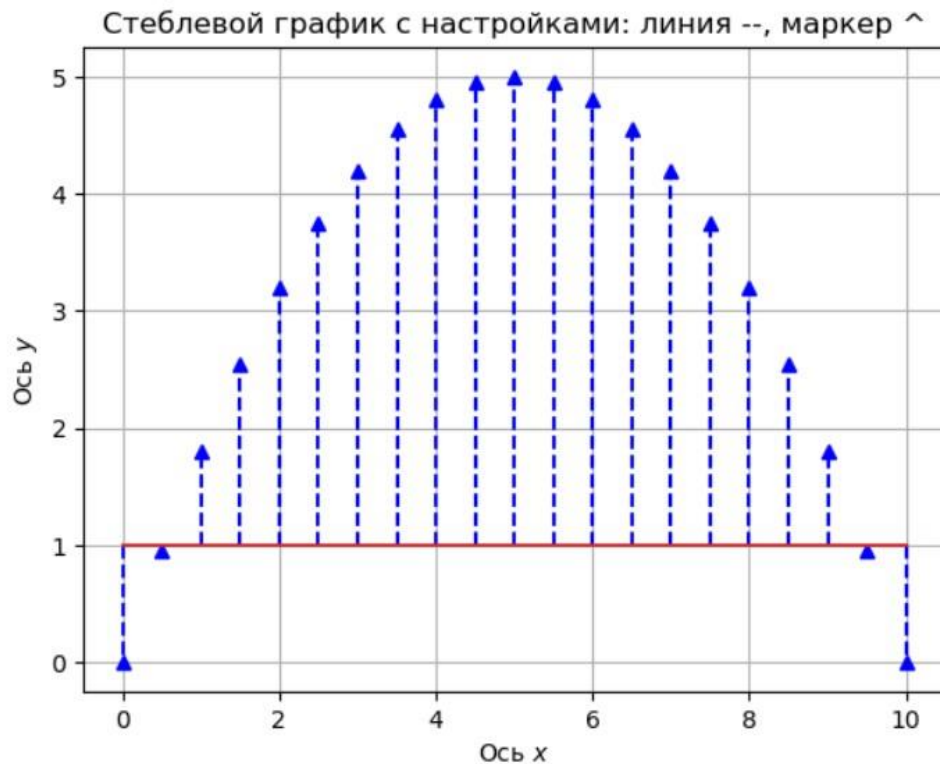


Рисунок 23 - Результат выполнения программы

24. `np.arange(0, 10.5, 0.5)` и `np.cos(x)` генерируют данные. `plt.scatter(x, y)` строит диаграмму рассеяния (точечный график). Оформляются заголовок, подписи осей и сетка. `plt.show()` выводит результат.

```
x = np.arange(0, 10.5, 0.5) y
```

```
= np.cos(x)
```

```
plt.scatter(x, y)
```

```
plt.title(r'Диаграмма рассеяния для $y = \cos(x)$')
plt.xlabel(r'Ось $x$') plt.ylabel(r'Ось $y$')
plt.grid(True) plt.show()
```

Результат выполнения:

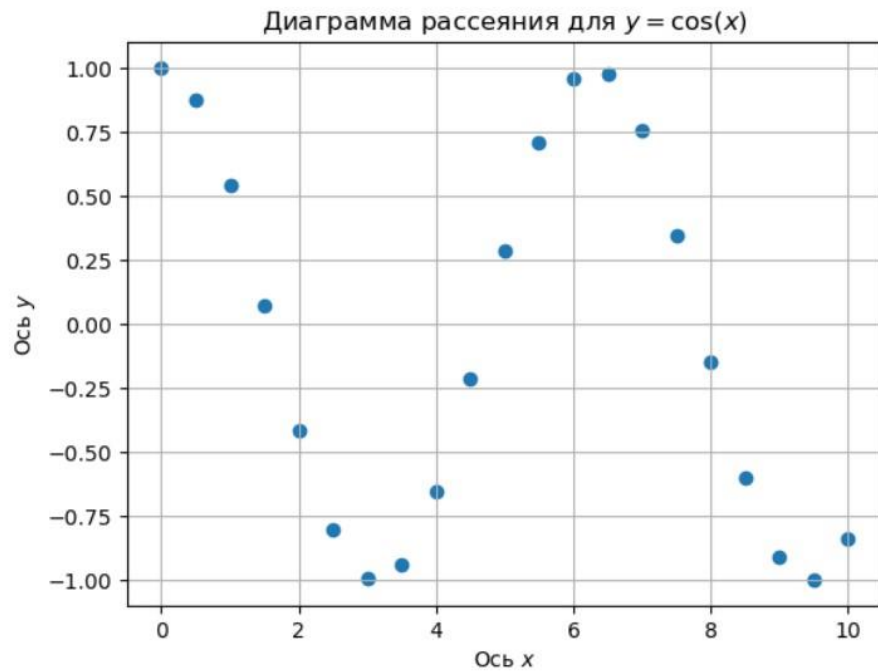


Рисунок 24 - Результат выполнения программы

25. `plt.scatter(x, y, s=80, c='r', marker='D', linewidths=2, edgecolors='b')`
 строит точечный график с размером точек 80, красным цветом,
 маркерами ромбами (D), толщиной обводки 2 и синим цветом края маркера.

```
x = np.arange(0, 10.5, 0.5) y
= np.cos(x)
```

```
plt.scatter(x, y, s=80, c='r', marker='D', linewidths=2, edgecolors='b')
plt.title(r'Диаграмма рассеяния с $y = \cos(x)$ и настройками маркера')
plt.xlabel(r'Ось $x$') plt.ylabel(r'Ось $y$') plt.grid(True) plt.show()
```

Результат выполнения:

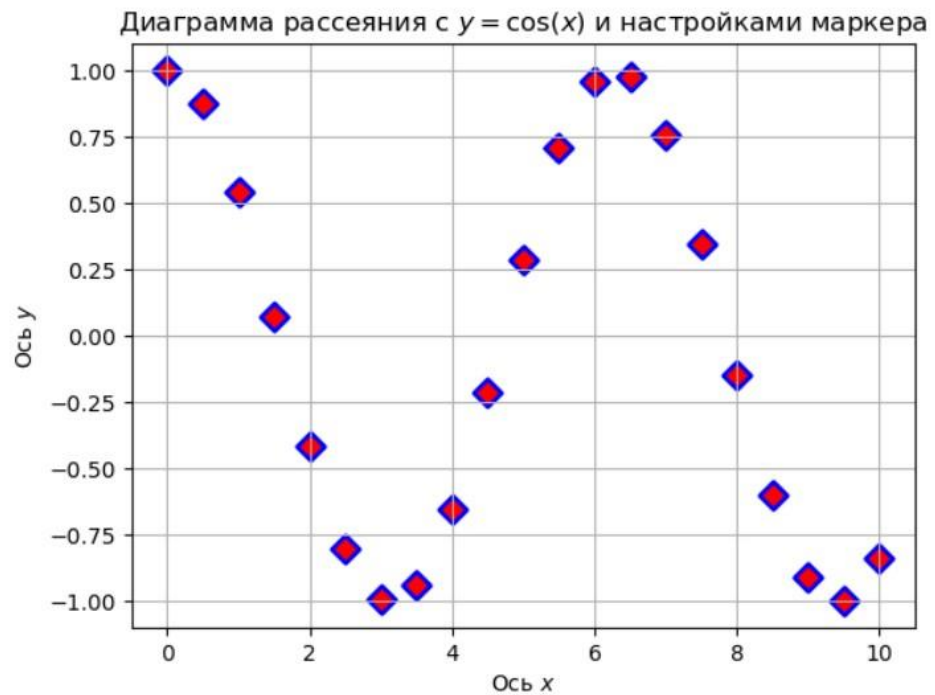


Рисунок 25 - Результат выполнения программы

26. `np.random.seed(123)` фиксирует случайность для воспроизводимости. Список `groups` содержит метки категорий `p0...p6`. `np.random.randint(3, 10, len(groups))` генерирует случайные значения от 3 до 10 для каждой категории. `plt.bar(groups, counts)` строит вертикальную столбчатую диаграмму. `plt.grid(True, axis='y')` включает сетку только по вертикальной оси. `np.random.seed(123)` `groups = [f'p{i}' for i in range(7)]` `counts = np.random.randint(3, 10, len(groups))`

```
plt.bar(groups, counts)
plt.title(r'Вертикальная столбчатая диаграмма для категорий $p_i$')
plt.xlabel(r'Категории') plt.ylabel(r'Значения') plt.grid(True, axis='y')
plt.show()
```

Результат выполнения:

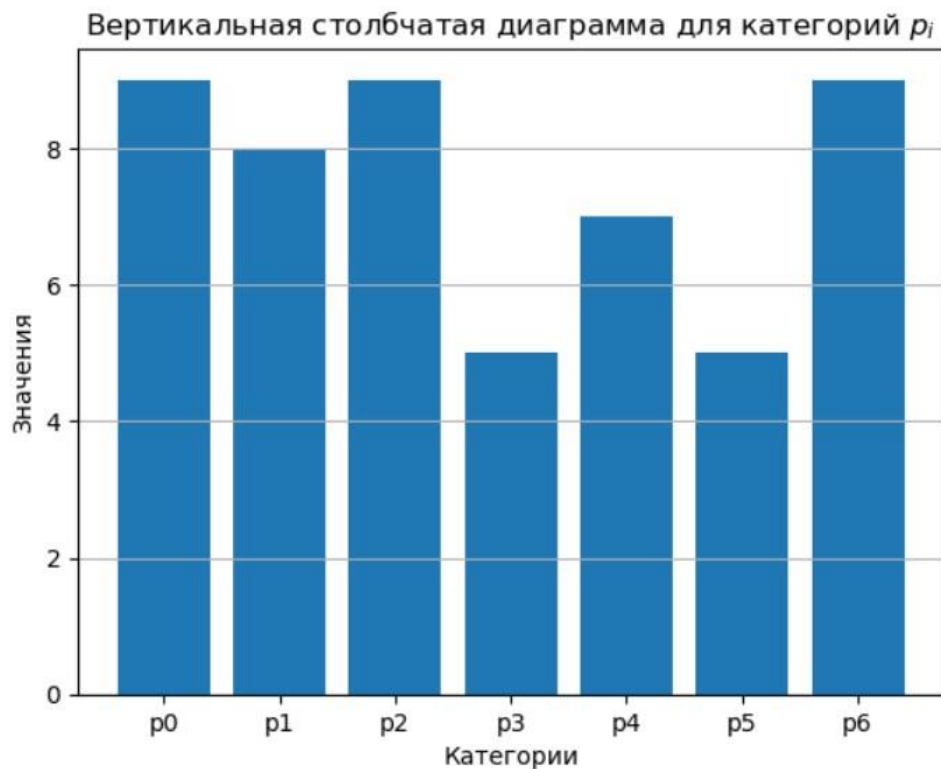


Рисунок 26 - Результат выполнения программы

27. `plt.barh(groups, counts)` строит горизонтальную столбчатую диаграмму. `plt.grid(True, axis='x')` включает сетку только по горизонтальной оси.

```
np.random.seed(123) groups = [f'p{i}' for i in
range(7)] counts = np.random.randint(3, 10,
len(groups))
```

```
plt.barh(groups, counts)
```

```
plt.title(r'Горизонтальная столбчатая диаграмма для категорий $p_i$')
```

```
plt.xlabel(r'Значения') plt.ylabel(r'Категории') plt.grid(True, axis='x')
```

```
plt.show()
```

Результат выполнения:

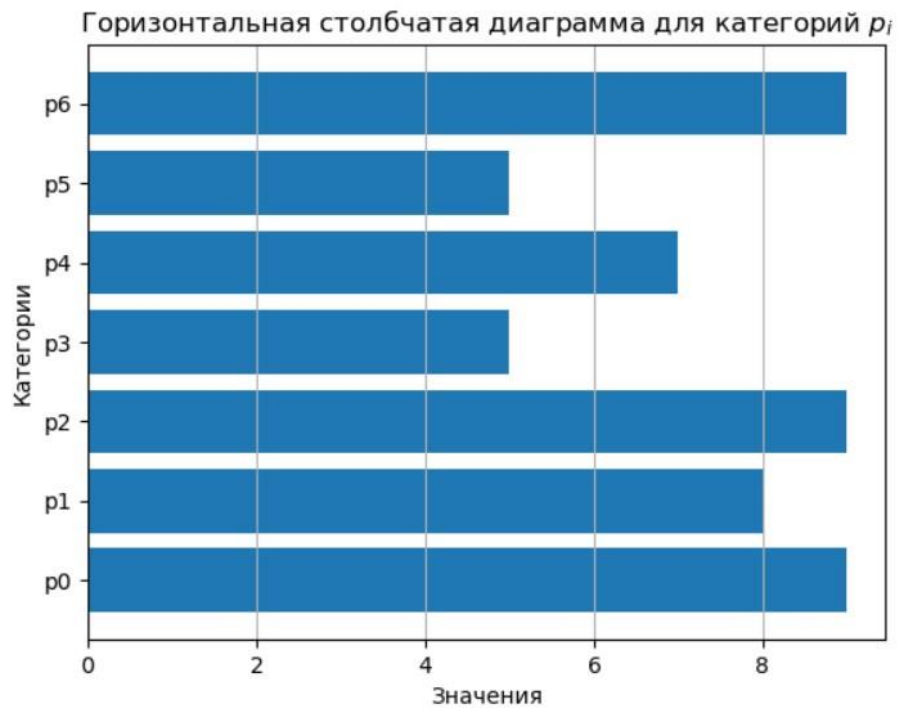


Рисунок 27 - Результат выполнения программы

28. Списки $g1$ и $g2$ содержат данные для двух групп. $width = 0.3$ задаёт ширину столбцов. $x = np.arange(len(cat_par))$ создаёт числовые позиции для категорий. $ax.bar(x - width/2, g1, width, label='g1')$ и $ax.bar(x + width/2, g2, width, label='g2')$ строят несколько столбцов со смещением для группировки. $ax.set_xticks(x)$ и $ax.set_xticklabels(cat_par)$ устанавливают метки категорий. $ax.legend()$ отображает легенду.

```
cat_par = [f'P{i}' for i in range(5)]
g1 = [10, 21, 34, 12, 27] g2 = [17,
15, 25, 21, 26] width = 0.3 x =
np.arange(len(cat_par))
```

```
fig, ax = plt.subplots() results1 = ax.bar(x -
width/2, g1, width, label='g1') results2 = ax.bar(x +
width/2, g2, width, label='g2')
ax.set_title(r'Групповая столбчатая диаграмма
для данных $g_1$ и
```



```

$g_2$') ax.set_xticks(x)
ax.set_xticklabels(cat_par)
ax.legend()
ax.set_xlabel(r'Категории')
ax.set_ylabel(r'Значения')
ax.grid(True, axis='y')
plt.show()

```

Результат выполнения:

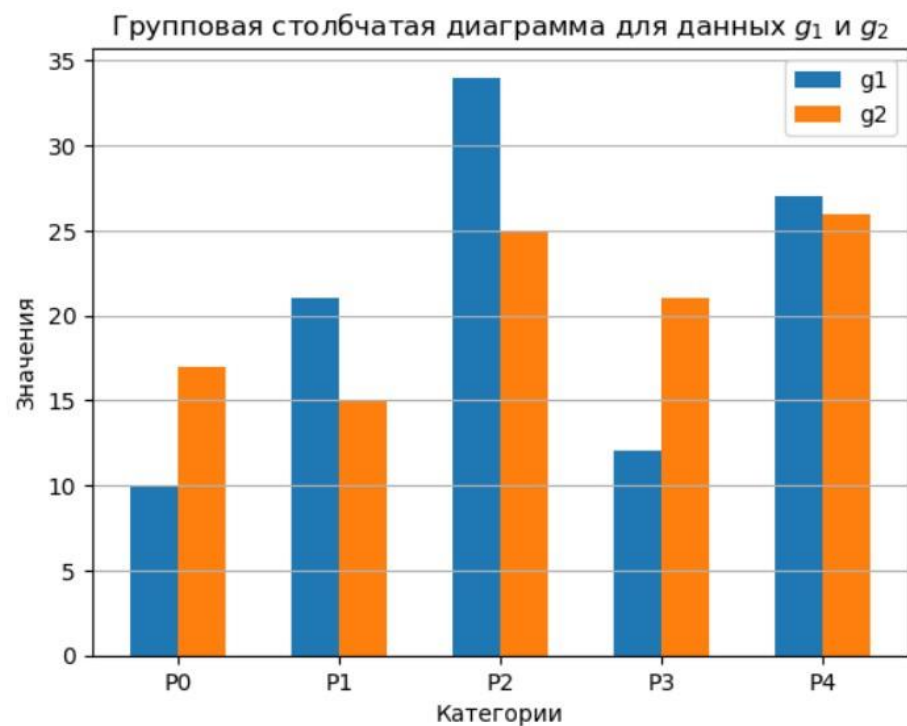


Рисунок 28 - Результат выполнения программы

29. `vals` — значения секторов, `labels` — подписи. `ax.pie(vals, labels=labels)` строит круговую диаграмму. `ax.axis("equal")` делает диаграмму круглой (равные пропорции осей). `ax.set_title()` задаёт заголовок.

```
vals = [24, 17, 53, 21, 35]
```

```
labels = ["Ford", "Toyota", "BMW", "AUDI", "Jaguar"]
```

```
fig, ax = plt.subplots() ax.pie(vals,
labels=labels) ax.axis("equal")
ax.set_title(r'Круговая диаграмма для автомобильных брендов')
plt.show()
```

Результат выполнения:

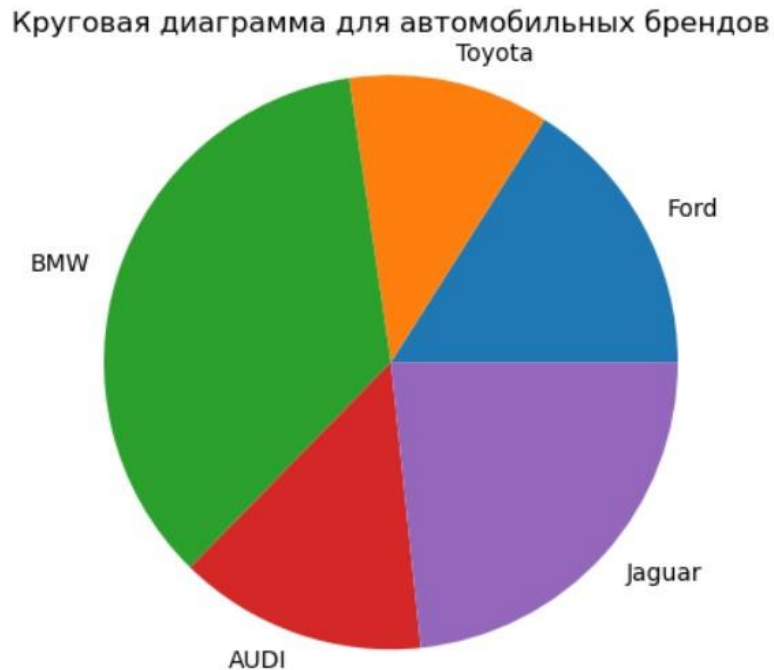


Рисунок 29 - Результат выполнения программы

30. `explode = (0.1, 0, 0.15, 0, 0)` задаёт вынос для первого и третьего секторов. `autopct='%1.1f%%'` отображает процентные значения с одним знаком после запятой. `shadow=True` добавляет тень. `wedgeprops={'lw': 1, 'ls': '-', 'edgecolor': "k"}` настраивает границы секторов (толщина 1, пунктир, чёрный цвет). `rotatelabels=True` поворачивает подписи.

```
vals = [24, 17, 53, 21, 35] labels = ["Ford", "Toyota",
"BMW", "AUDI", "Jaguar"] explode = (0.1, 0, 0.15, 0,
0)
```

```
fig, ax = plt.subplots()
```

```

ax.pie(vals, labels=labels, autopct='%1.1f%%', shadow=True,
explode=explode, wedgeprops={'lw': 1, 'ls': '--', 'edgecolor': "k"}, rotatelabels=True)
ax.axis("equal")

ax.set_title(r'Круговая диаграмма с выносом сектора ( $\mathbf{BMW}$ )
и настройками') plt.show()

```

Результат выполнения:

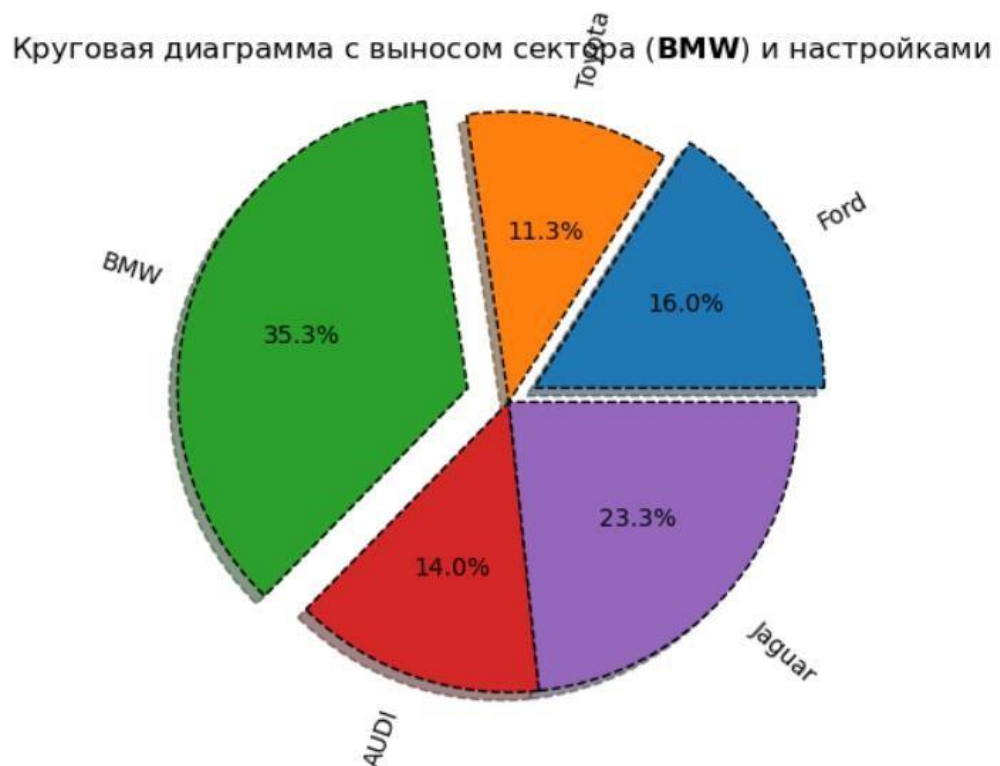


Рисунок 30 - Результат выполнения программы

31. `ax.pie(data.sum(axis=1), radius=1, colors=b_colors, wedgeprops=dict(width=offset, edgecolor='w'))` строит внешнее кольцо диаграммы с радиусом 1 и шириной колец `offset`. `ax.pie(data.flatten(), radius=1-offset, colors=sm_colors, wedgeprops=dict(width=offset, edgecolor='w'))` строит внутреннее кольцо с меньшим радиусом. В результате получается многослойная кольцевая диаграмма для иерархических данных.

```
fig, ax = plt.subplots()
```

```
offset = 0.4
```

```
data = np.array([[5, 10, 7], [8, 15, 5], [11, 9, 7]]) cmap  
= plt.get_cmap("tab20b")
```

```
b_colors = cmap(np.array([0, 8, 12])) sm_colors =  
cmap(np.array([1, 2, 3, 9, 10, 11, 13, 14, 15]))
```

```
ax.pie(data.sum(axis=1), radius=1, colors=b_colors,  
wedgeprops=dict(width=offset, edgecolor='w'))  
ax.pie(data.flatten(), radius=1-offset, colors=sm_colors,  
wedgeprops=dict(width=offset, edgecolor='w'))  
ax.set_title(r'Кольцевая диаграмма для иерархических данных')  
plt.show()
```

Результат выполнения:

Кольцевая диаграмма для иерархических данных



Рисунок 31 - Результат выполнения программы

32. `wedgeprops=dict(width=0.5)` задаёт ширину кольца 0.5, превращая обычную круговую диаграмму в кольцевую.

```
vals = [24, 17, 53, 21, 35]
```

```
labels = ["Ford", "Toyota", "BMW", "AUDI", "Jaguar"]
```

```
fig, ax = plt.subplots() ax.pie(vals, labels=labels,
```

```
wedgeprops=dict(width=0.5)) ax.axis("equal")
```

```
ax.set_title(r'Кольцевая диаграмма для автомобильных брендов')
```

```
plt.show()
```

Результат выполнения:



Рисунок 32 - Результат выполнения программы

33. `requests.get(url)` загружает изображение по URL.

`Image.open(BytesIO(response.content))` открывает изображение из полученных данных. `plt.imshow(img)` отображает изображение. Оформляются заголовок, подписи осей (в пикселях) и сетка.

```
from PIL import Image
```

```
import requests from io
```

```
import BytesIO
```

```

response
requests.get('https://avatars.mds.yandex.net/i?id=c068d855b728a7d802773ebebfl2
5199_1-5176581-images-thumbs&n=13') img =
Image.open(BytesIO(response.content))

plt.imshow(img)
plt.title(r'Отображение изображения')
plt.xlabel(r'Ось $x$ (пиксели)') plt.ylabel(r'Ось
$y$ (пиксели)') plt.grid(True, linestyle='--',
alpha=0.5) plt.show()

```

Результат выполнения:

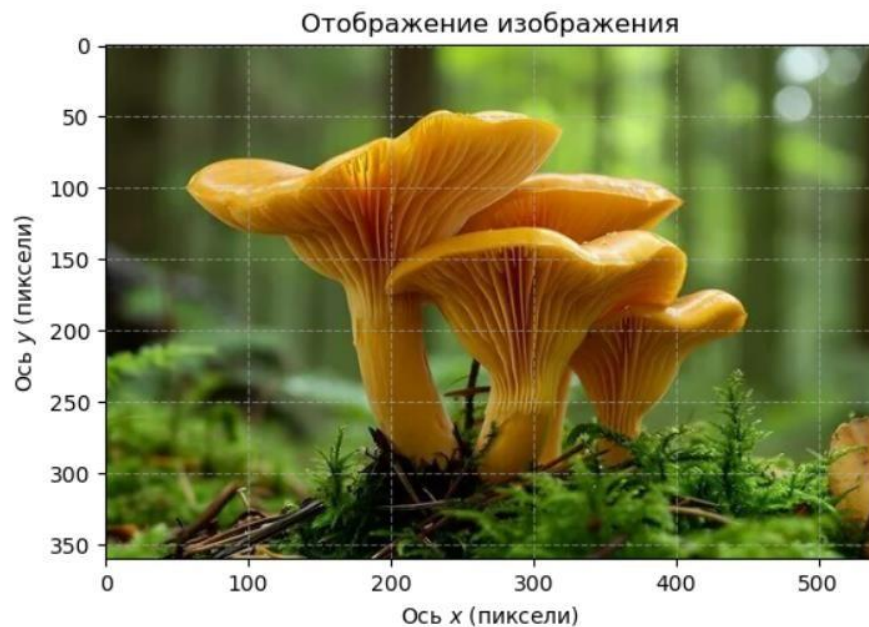


Рисунок 33 - Результат выполнения программы

34. `np.random.randn(25, 25)` генерирует массив 25×25 случайных чисел из нормального распределения. `plt.imshow(data)` отображает тепловую карту. Оформляются заголовок, подписи осей и сетка.

```
np.random.seed(19680801)
```

```

data = np.random.randn(25, 25)
plt.imshow(data) plt.title(r'Тепловая карта
случайных данных') plt.xlabel(r'Ось $x$
(пиксели)') plt.ylabel(r'Ось $y$ (пиксели)')
plt.grid(True, linestyle='--', alpha=0.5)
plt.show()

```

Результат выполнения:

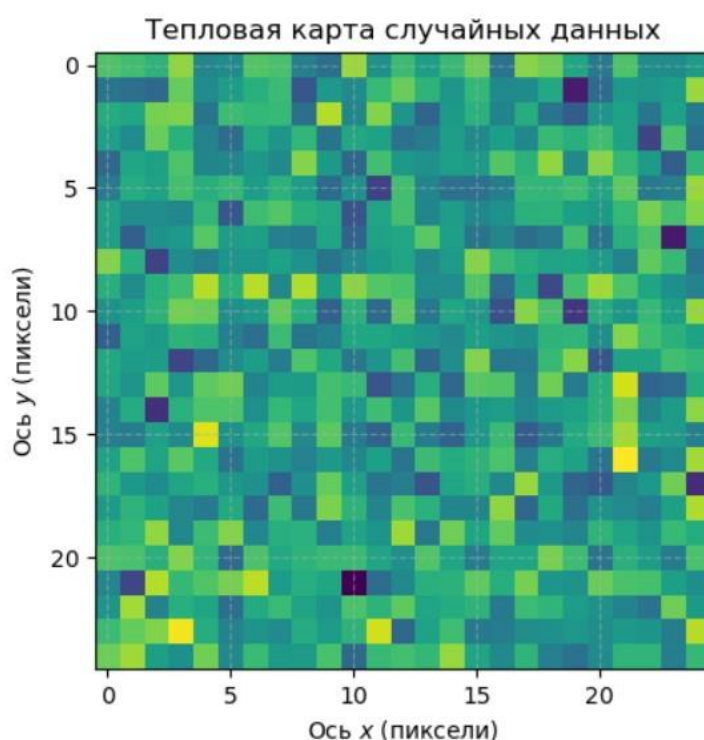


Рисунок 34 - Результат выполнения программы

35. `plt.subplots(1, 2)` создаёт два подокна. Первое: `imshow(data, cmap='winter', vmin=-1, vmax=1, origin='lower')` с цветовой картой `winter`, диапазоном значений от -1 до 1 и началом координат внизу. `fig.colorbar(p1, ax=axes[0])` добавляет цветовую шкалу. Второе: `imshow(data, cmap='plasma', interpolation='gaussian', origin='lower', extent=(0, 30, 0, 30))` с интерполяцией Gaussian и расширенными границами осей от 0 до 30.

```
fig, axes = plt.subplots(1, 2, figsize=(10, 3), constrained_layout=True)
```

```

p1 = axs[0].imshow(data, cmap='winter', aspect='equal', vmin=-1, vmax=1,
origin='lower') fig.colorbar(p1, ax=axs[0])

axs[0].set_title(r'Тепловая карта с colormap')
axs[0].set_xlabel(r'Ось $x$ (пиксели)')
axs[0].set_ylabel(r'Ось $y$ (пиксели)')
axs[0].grid(True, linestyle='--', alpha=0.3)

p2 = axs[1].imshow(data, cmap='plasma', aspect='equal',
interpolation='gaussian', origin='lower', extent=(0, 30, 0, 30)) fig.colorbar(p2,
ax=axs[1])

axs[1].set_title(r'Тепловая карта с colormap и интерполяцией')
axs[1].set_xlabel(r'Ось $x$ (0–30)') axs[1].set_ylabel(r'Ось $y$
(0–30)') axs[1].grid(True, linestyle='--', alpha=0.3)

plt.show()

```

Результат выполнения:

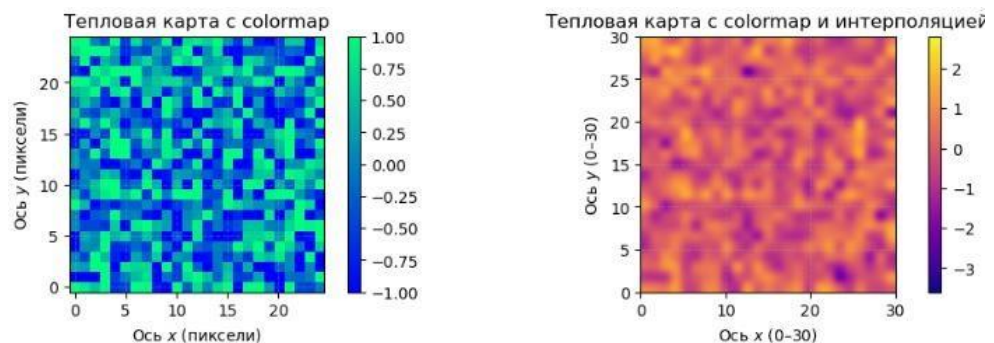


Рисунок 35 - Результат выполнения программы

36. `np.random.rand(5, 7)` генерирует случайную матрицу 5×7 .
`plt.pcolormesh(data, cmap='plasma', edgecolors='k', shading='flat')` строит

сетчатую тепловую карту с чёрными границами ячеек и плоской заливкой. `plt.colorbar()` добавляет цветовую шкалу.

```
np.random.seed(123) data = np.random.rand(5, 7)
plt.pcolormesh(data, cmap='plasma', edgecolors='k', shading='flat')
plt.title(r'Диаграмма со случайными данными') plt.xlabel(r'Ось
$x$ (индексы столбцов)') plt.ylabel(r'Ось $y$ (индексы строк)')

plt.grid(True, linestyle='--', alpha=0.5)
plt.colorbar(label=r'Значения') plt.show()
```

Результат выполнения:

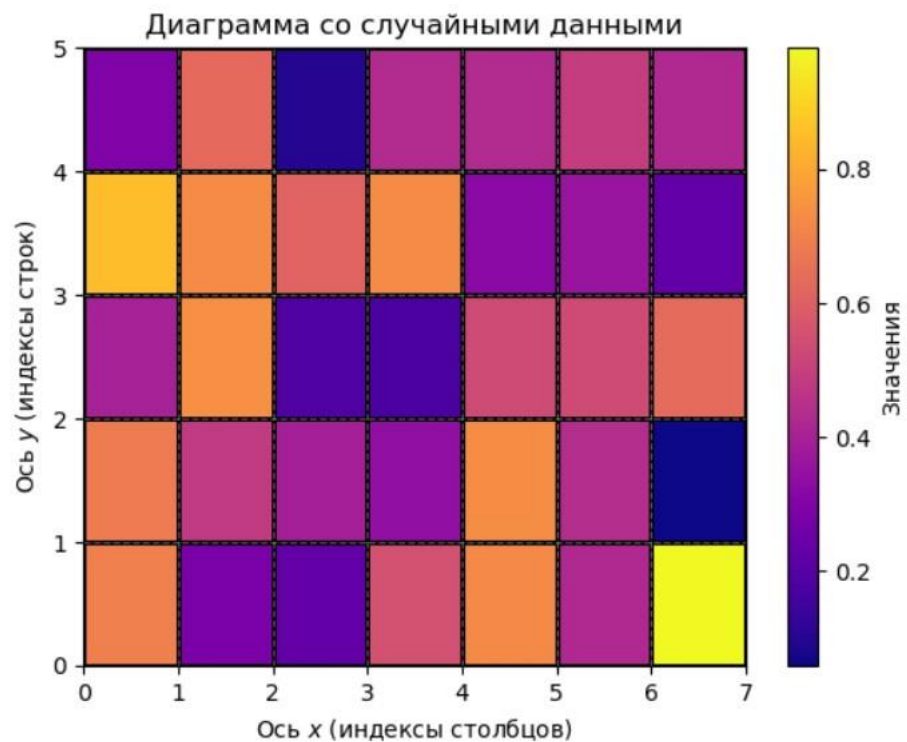


Рисунок 36 - Результат выполнения программы

37. `fig.add_subplot(111, projection='3d')` создаёт 3D-оси. `ax.plot(x, y, z, label='parametric curve')` строит параметрическую кривую: $x = t$, $y = t$, $z = \cos(t)$. Методами `set_xlabel`, `set_ylabel`, `set_zlabel` подписываются оси. `ax.grid(True)` включает сетку.

```

x = np.linspace(-np.pi, np.pi, 50)
y = x
z = np.cos(x)

fig = plt.figure()
ax = fig.add_subplot(111, projection='3d')
ax.plot(x, y, z, label='parametric curve')
ax.set_title(r'Параметрическая кривая в 3D: $x = t, y = t, z = \cos(t)$')
ax.set_xlabel(r'Ось $x$')
ax.set_ylabel(r'Ось $y$')
ax.set_zlabel(r'Ось $z$')
ax.grid(True)
plt.show()

```

Результат выполнения:

Параметрическая кривая в 3D: $x = t, y = t, z = \cos(t)$

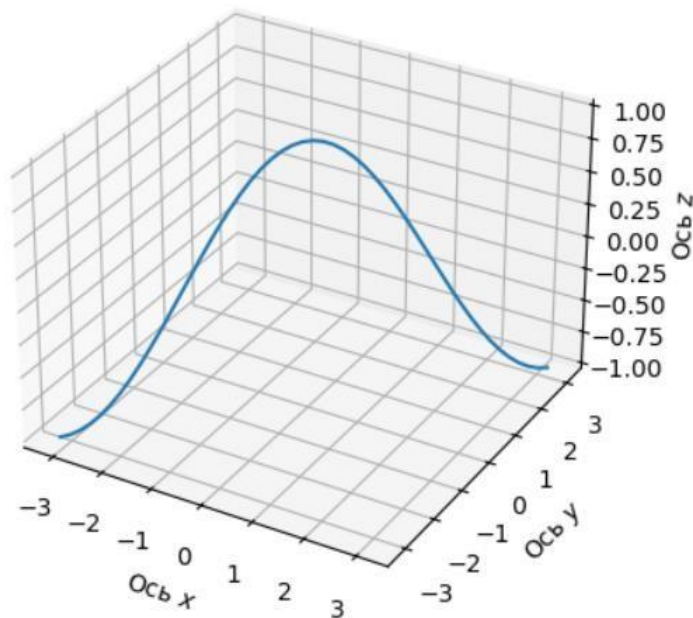


Рисунок 37 - Результат выполнения программы

38. Генерация 40 случайных точек для 3D-диаграммы рассеяния. Функция `np.random.seed(123)` фиксирует генератор случайных чисел для

воспроизводимости результатов. Массивы x и z генерируются в диапазоне от -5 до 5, y – от 0 до 10, s (размер маркеров) – от 10 до 100. fig.add_subplot(111, projection='3d') создаёт 3D-оси, ax.scatter() строит диаграмму рассеяния с разным размером точек. Методами set_title, set_xlabel, set_ylabel, set_zlabel и grid(True) оформляются заголовок, подписи осей и сетка. plt.show() выводит результат.

```
np.random.seed(123) x =  
np.random.randint(-5, 5, 40) y =  
np.random.randint(0, 10, 40) z =  
np.random.randint(-5, 5, 40) s =  
np.random.randint(10, 100, 40)  
  
fig = plt.figure() ax =  
fig.add_subplot(111, projection='3d')  
  
ax.scatter(x, y, z, s=s)  
  
ax.set_title(r'3D диаграмма рассеяния со случайными данными')  
ax.set_xlabel(r'Ось $x$') ax.set_ylabel(r'Ось $y$')  
ax.set_zlabel(r'Ось $z$') ax.grid(True)  
  
plt.show()
```

Результат выполнения:

3D диаграмма рассеяния со случайными данными

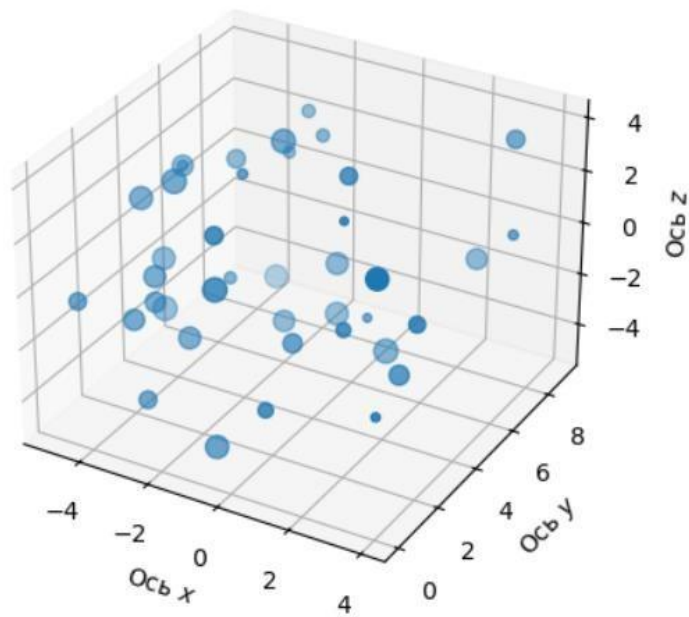


Рисунок 38 - Результат выполнения программы

36. Построение каркасной 3D-поверхности сферы. Функции `np.mgrid` создают сетку углов: u от 0 до 2π с 40 делениями (азимутальный угол), v от 0 до π с 20 делениями (полярный угол). Параметрические уравнения $x = \cos(u) \cdot \sin(v)$, $y = \sin(u) \cdot \sin(v)$, $z = \cos(v)$ описывают сферу. `ax.plot_wireframe(x, y, z)` строит каркасную поверхность, где линии сетки отображаются без заливки. `ax.legend()` добавляет легенду с меткой "Сфера".

Оформляются заголовок, подписи осей и сетка.

```
u, v = np.mgrid[0:2*np.pi:40j, 0:np.pi:20j]
```

```
x = np.cos(u) * np.sin(v) y = np.sin(u) *  
np.sin(v) z = np.cos(v)
```

```
fig = plt.figure() ax = fig.add_subplot(111,  
projection='3d') ax.plot_wireframe(x, y, z,  
label="Сфера") ax.legend()
```

```
ax.set_title(r'3D поверхность: сфера')
```

```
ax.set_xlabel(r'Ось $x$')
```

```
ax.set_ylabel(r'Ось $y$')
ax.set_zlabel(r'Ось $z$') ax.grid(True)

plt.show()
```

Результат выполнения:

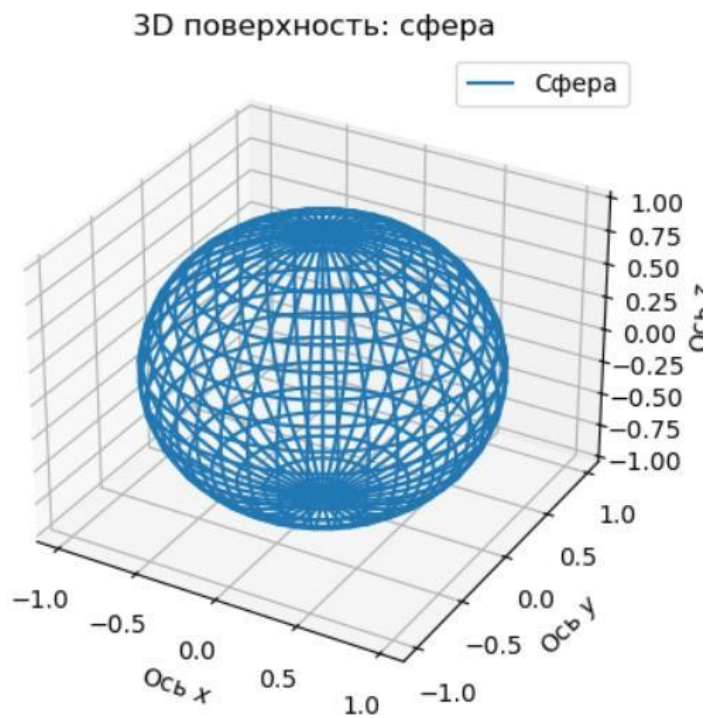


Рисунок 39 - Результат выполнения программы

39. Построение залитой 3D-поверхности сферы с цветовой картой. Сетка углов создаётся с меньшим разрешением: u – 20 делений, v – 10 делений для более быстрой визуализации. `ax.plot_surface(x, y, z, cmap='inferno')` строит поверхность с заливкой, где параметр `cmap='inferno'` задаёт цветовую карту – от чёрного/фиолетового (низкие значения) до жёлтого/белого (высокие значения), что позволяет визуально оценить высоту z . Оформляются заголовок, подписи осей и сетка.

```
u, v = np.mgrid[0:2*np.pi:20j, 0:np.pi:10j]
x = np.cos(u) * np.sin(v) y = np.sin(u) *
np.sin(v) z = np.cos(v)
```

```

fig = plt.figure() ax = fig.add_subplot(111,
projection='3d') ax.plot_surface(x, y, z,
cmap='inferno') ax.set_title(r'3D поверхность:
сфера с colormap') ax.set_xlabel(r'Ось $x$')
ax.set_ylabel(r'Ось $y$') ax.set_zlabel(r'Ось $z$')
ax.grid(True)

plt.show()

```

Результат выполнения:

3D поверхность: сфера с colormap

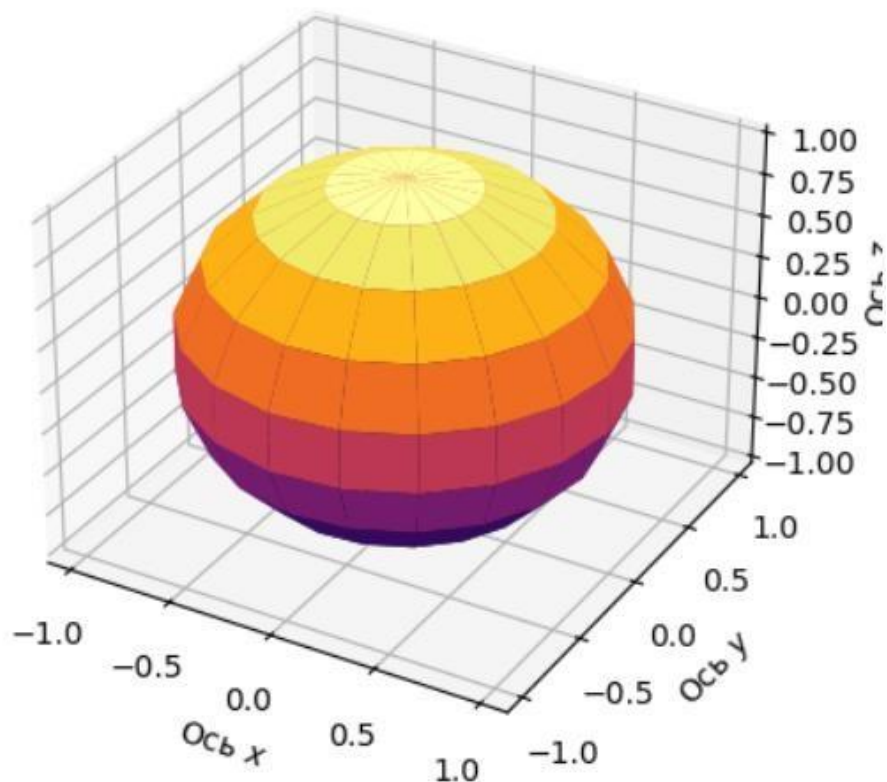


Рисунок 40 - Результат выполнения программы

Задания.

Задание 1. Построение простого графика.

Напишите код, который строит график функции $y = x^2$ на интервале $[-10, 10]$. Добавьте заголовок, подписи осей и сетку.

Решение: Функция `np.linspace(-10, 10, 50)` создаёт 50 равномерно распределённых точек на интервале $[-10, 10]$. Генератор списка `[i**2 for i in x]` вычисляет значения квадратичной функции. `plt.plot(x, y)` строит график. `plt.title()`, `plt.xlabel()`, `plt.ylabel()` оформляют заголовок и подписи осей с поддержкой LaTeX. `plt.grid(True)` включает сетку. `plt.show()` выводит результат.

Листинг:

```
x = np.linspace(-10,10,50)
y = [i**2 for i in x]

plt.title(r'График функции $y = x^2$ на интервале $[-10, 10]$')
plt.xlabel(r'$x$', fontsize=12) plt.ylabel(r'$y$', fontsize=12)
plt.grid(True) plt.plot(x,y) plt.show()
```

Результат выполнения:

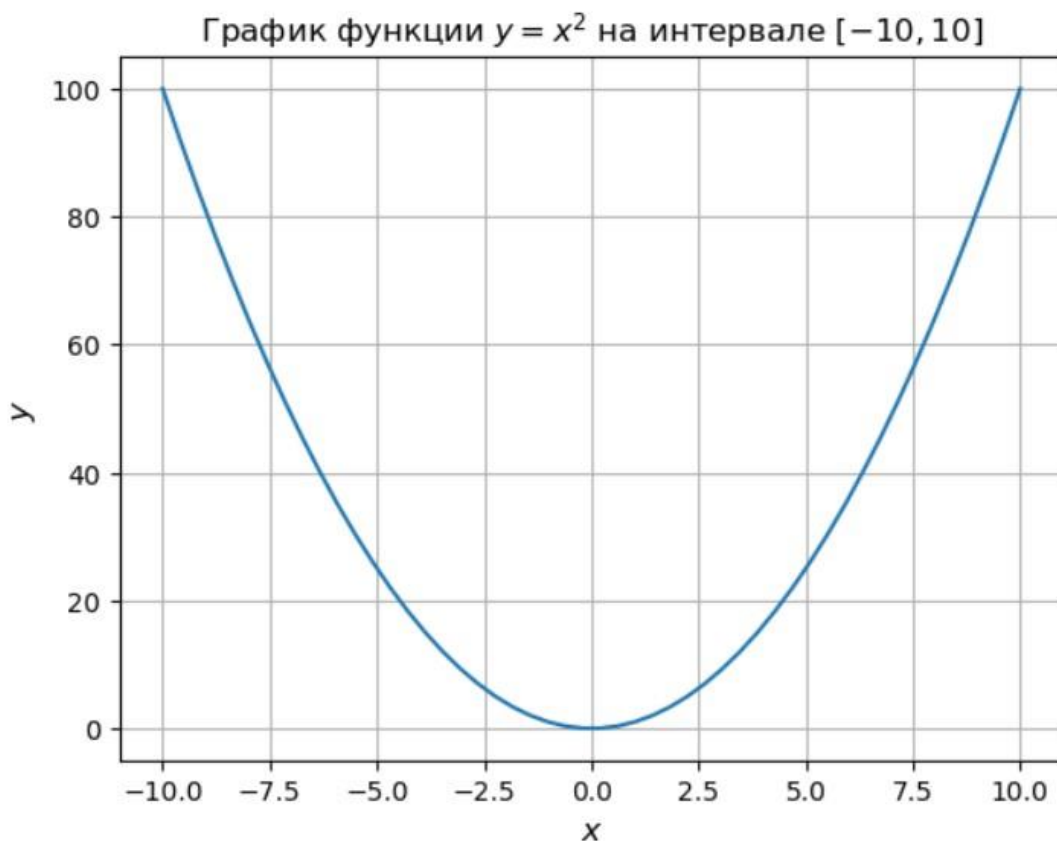


Рисунок 41 - Результат выполнения программы

Задание 2. Настройка стилей и цветов.

Постройте три линии на одном графике:

1. $y = x$ (синяя, пунктирная линия),
2. $y = x^2$ (зеленая, штрих-пунктирная линия),
3. $y = x^3$ (красная, сплошная линия).

Добавьте легенду и сделайте оси одинакового масштаба.

Решение: `np.linspace(-1, 1)` создаёт массив x на интервале $[-1, 1]$. Для каждого x вычисляются $y_1 = x$ (линейная), $y_2 = x^2$ (квадратичная), $y_3 = x^3$ (кубическая). `plt.plot(x, y1, '--b')` строит синюю пунктирную линию, `plt.plot(x, y2, '-.g')` – зелёную штрих-пунктирную, `plt.plot(x, y3, '-r')` – красную сплошную. `plt.legend()` добавляет легенду. `plt.axis('equal')` делает одинаковый масштаб осей.

Листинг:

```
x = np.linspace(-1,1)
y1 = [i for i in x] y2
= [i**2 for i in x] y3
= [i**3 for i in x]

plt.plot(x, y1, '--b', label=r"$y_1$")
plt.plot(x, y2, '-.g', label=r"$y_2$") plt.plot(x,
y3, '-r', label=r"$y_3$")

plt.title(r"Графики функций $y_1(x)$, $y_2(x)$, $y_3(x)$", fontsize=14)
plt.xlabel(r"$x$", fontsize=12) plt.ylabel(r"$y$", fontsize=12)
plt.grid(True) plt.legend() plt.axis('equal') plt.show()
```

Результат выполнения:

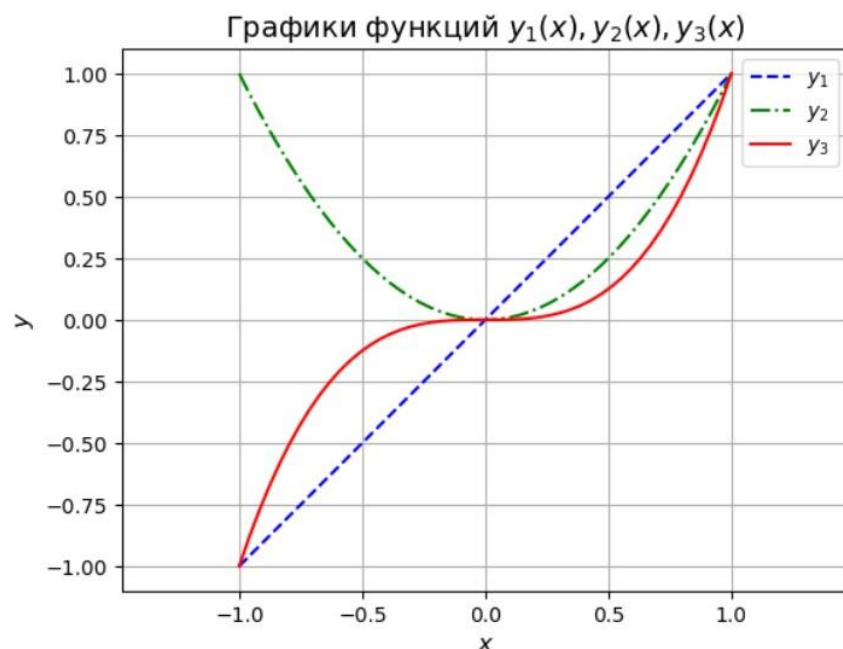


Рисунок 42 - Результат выполнения программы

Задание 3. Использование различных типов графиков.

Сгенерируйте 50 случайных точек и постройте диаграмму рассеяния (scatter plot), где цвет точек зависит от их координаты по оси x, а размер точек зависит от координаты по оси y.

Решение: `np.random.seed(42)` фиксирует генератор случайных чисел для воспроизводимости. `np.random.rand(50) * 10` генерирует 50 случайных координат x и y от 0 до 10. `plt.scatter(x, y, c=x, s=y*50)` строит диаграмму рассеяния, где параметр `c=x` задаёт цвет точки в зависимости от её координаты x, а `s=y*50` – размер точки пропорционально координате y. `edgecolors='black'` добавляет чёрную обводку маркеров.

Листинг:

```
np.random.seed(42) n = 50
x = np.random.rand(n) * 10
y = np.random.rand(n) * 10
plt.scatter(x, y, c=x,
s=y*50,
edgecolors='black',
linewidth=0.5)
plt.title(r"Диаграмма
рассеяния: цвет от $x$,
размер от $y$",
fontsize=14)
plt.xlabel(r"$x$",
fontsize=12)
plt.ylabel(r"$y$",
fontsize=12) plt.grid(True)
plt.show()
```

Результат выполнения:

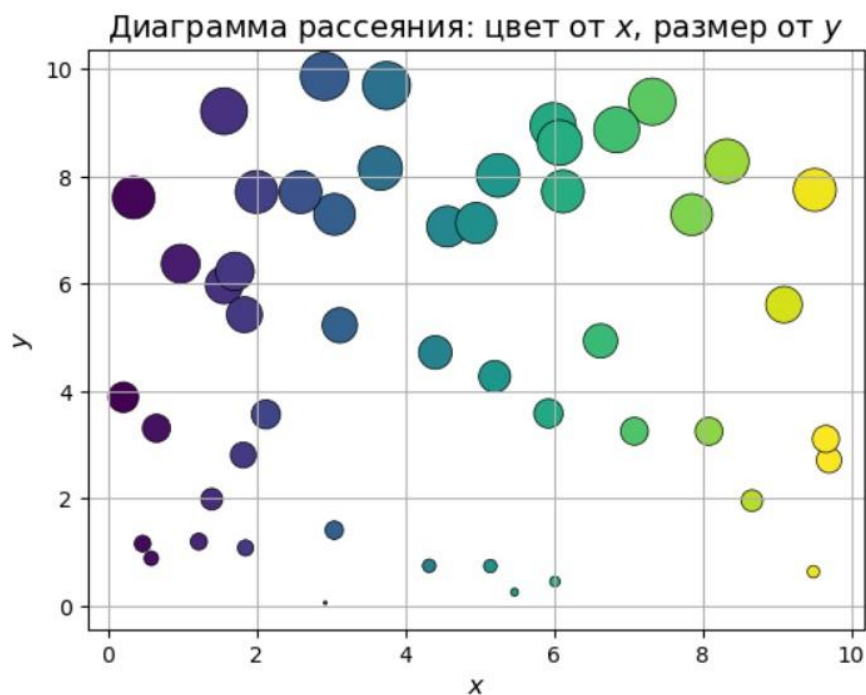


Рисунок 43 - Результат выполнения программы

Задание 4. Гистограмма распределения.

Сгенерируйте 1000 случайных чисел из нормального распределения с параметрами $\mu = 0$, $\sigma = 1$ и постройте их гистограмму с 30 бинами. Добавьте вертикальную линию в среднем значении.

Решение: `np.random.seed(123)` фиксирует случайность. `np.random.normal(0, 1, 1000)` генерирует 1000 чисел из нормального распределения со средним $\mu=0$ и стандартным отклонением $\sigma=1$. `plt.hist(data, bins=30)` строит гистограмму с 30 интервалами, `color='blue'` задаёт цвет столбцов, `edgecolor='black'` – обводку. `plt.axvline(np.mean(data), color='red', linestyle='--')` добавляет вертикальную пунктирную красную линию на уровне среднего значения.

Листинг:

```
np.random.seed(123) data =
np.random.normal(0, 1, 1000)
```

```
plt.hist(data, bins=30, color='blue', edgecolor='black', alpha=0.7)
plt.axvline(np.mean(data), color='red', linestyle='--', linewidth=2)
plt.title(r'Гистограмма нормального распределения  $\mu = 0$ ,  $\sigma = 1$ ',
fontsize=14) plt.xlabel(r'Значение',
fontsize=12) plt.ylabel(r'Частота',
fontsize=12) plt.grid(True)
plt.show()
```

Результат выполнения:

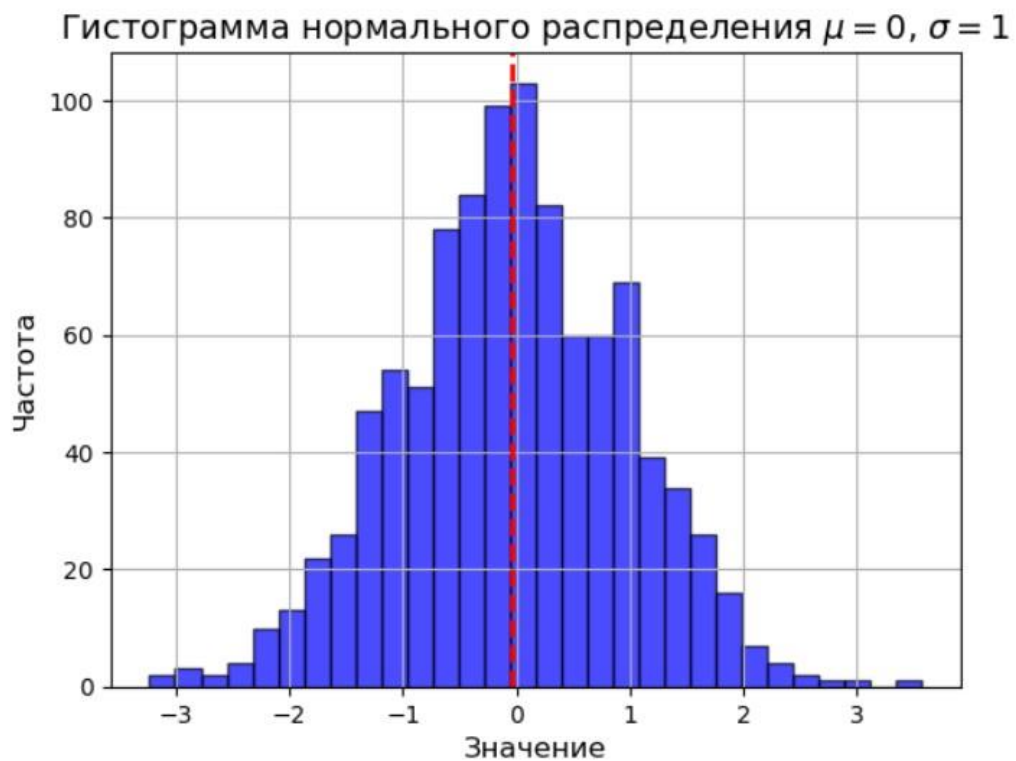


Рисунок 44 - Результат выполнения программы

Задание 5. Столбчатая диаграмма.

Создайте столбчатую диаграмму, которая показывает количество студентов, получивших оценки:

1. «Отлично» - 20 человек,
2. «Хорошо» - 35 человек,

3. «Удовлетворительно» - 30 человек, 4. «Неудовлетворительно»
- 15 человек.

Добавьте подписи и заголовок.

Решение: создаются списки `grade` (категории оценок) и `student` (количество студентов). `plt.bar(grade, student)` строит столбчатую диаграмму, где каждому названию оценки соответствует столбец высотой, равной числу студентов. `color='skyblue'` задаёт цвет столбцов, `edgecolor='black'` – чёрную обводку. Оформляются заголовок и подписи осей.

Листинг:

```
grade      =      ['Отлично',      'Хорошо',      'Удовлетворительно',  
'Неудовлетворительно']  
student  
= [20, 35, 30, 15]
```

```
plt.bar(grade, student, color='skyblue', edgecolor='black')  
plt.title('Распределение оценок студентов', fontsize=14)  
plt.xlabel('Оценка', fontsize=12) plt.ylabel('Количество  
студентов', fontsize=12) plt.grid(True) plt.show()
```

Результат выполнения:

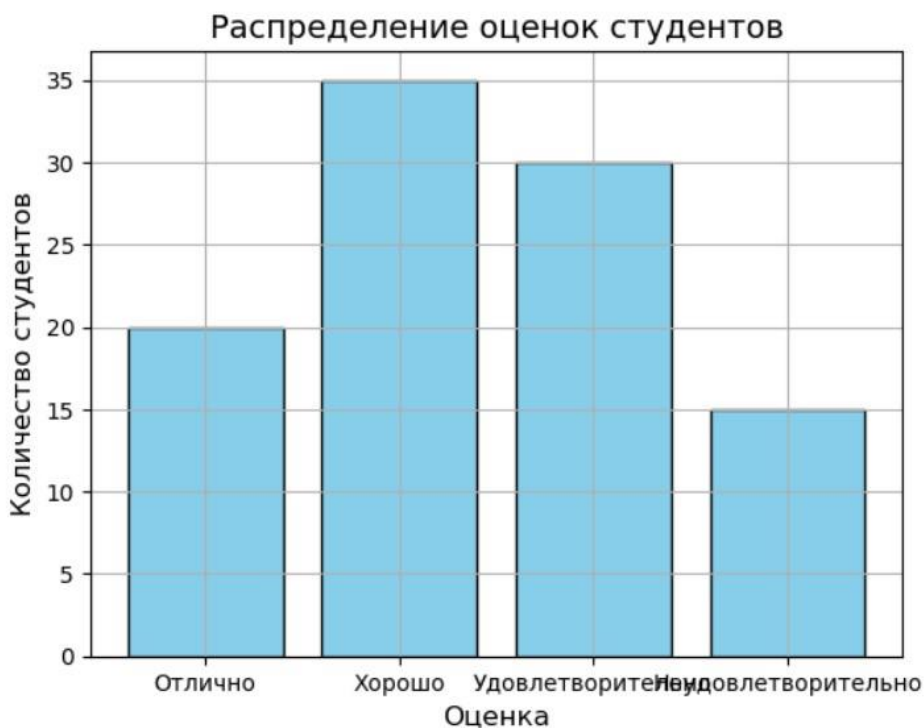


Рисунок 45 - Результат выполнения программы

Задание 6. Круговая диаграмма.

Используя данные предыдущей задачи, постройте круговую диаграмму с процентными подписями секторов.

Решение: используются те же данные `grade` и `student`. `plt.pie(student, labels=grade, autopct='%1.1f%%')` строит круговую диаграмму, где `autopct='%1.1f%%'` отображает проценты с одним знаком после запятой. `colors` задаёт цвета секторов, `explode=(0.05, 0, 0, 0.05)` выносит первый и последний сектор, `shadow=True` добавляет тень, `startangle=90` поворачивает диаграмму для лучшего вида. `plt.axis('equal')` делает круг круглым.

Листинг:

```
grade      =      ['Отлично',      'Хорошо',      'Удовлетворительно',
'Неудовлетворительно']
student = [20, 35,
30, 15]
colors = ['green', 'yellow',
'orange', 'red']
explode = (0.05, 0, 0,
0.05)
```

```
plt.pie(student, labels=grade, autopct='%1.1f%%', colors=colors,
explode=explode, shadow=True, startangle=90)
```

```
plt.title(r'Распределение оценок студентов', fontsize=14)
plt.axis('equal') plt.show()
```

Результат выполнения:



Рисунок 46 - Результат выполнения программы

Задание 7. Трехмерный график поверхности.

Используя `mpl_toolkits.mplot3d`, постройте 3D-график функции $z = \sin(\sqrt{x^2+y^2})$ на сетке значений x, y в диапазоне $[-5; 5]$.

Решение: `np.linspace(-5, 5, 100)` создаёт 100 точек по каждому измерению. `np.meshgrid(x, y)` образует двумерные сетки X и Y . Функция $Z = \sin(\sqrt{x^2+y^2})$ вычисляет значения. `fig.add_subplot(111, projection='3d')` создаёт 3D-оси. `ax.plot_surface(X, Y, Z, cmap='plasma')` строит заливную поверхность с цветовой картой `plasma`. Оформляется заголовок.

Листинг:

```
x = np.linspace(-5, 5, 100) y
= np.linspace(-5, 5, 100)
X, Y = np.meshgrid(x, y)
Z = np.sin(np.sqrt(X**2 + Y**2))

fig = plt.figure() ax =
fig.add_subplot(111, projection='3d')
ax.plot_surface(X, Y, Z, cmap='plasma')
ax.set_title(r'$z = \sin(\sqrt{x^2 + y^2})$') plt.show()
```

Результат выполнения:

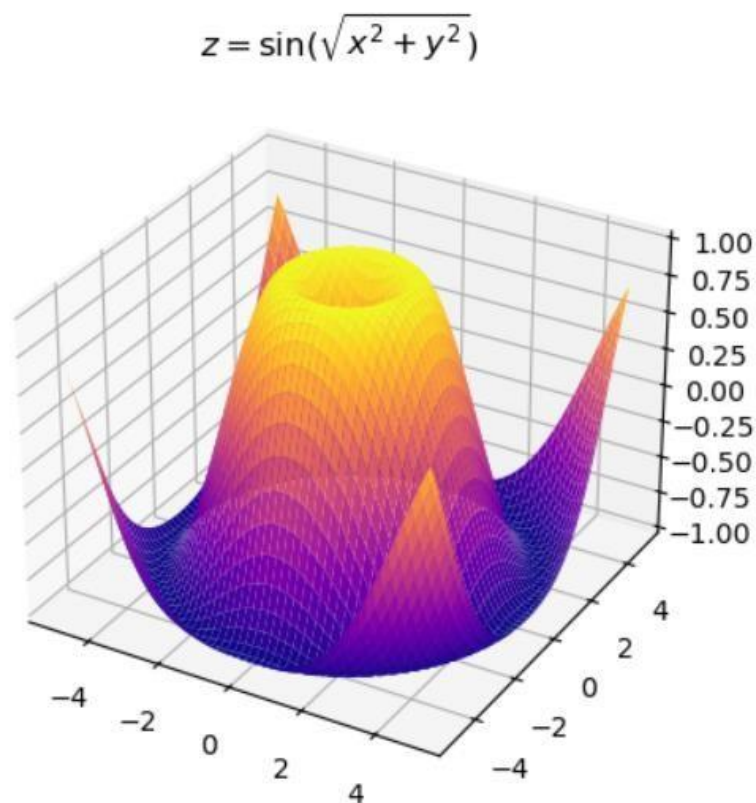


Рисунок 47 - Результат выполнения программы

Задание 8. Множественные подграфики (subplots).

Постройте четыре графика в одной фигуре (2x2 сетка):

1. Линейный график
2. Парабола
3. Синус
4. Косинус

Добавьте заголовки к каждому подграфику.

Решение: `fig, axs = plt.subplots(2, 2, figsize=(10, 8))` создаёт фигуру и сетку осей 2×2 . Обращение к подокнам осуществляется через индексы `axs[0,0]`, `axs[0,1]`, `axs[1,0]`, `axs[1,1]`. В каждом подокне методом `.plot()` строится график с разными цветами: синий для $y=x$, красный для $y=x^2$, зелёный для $\sin(x)$, пурпурный для $\cos(x)$. Методы `.set_title()`, `.set_xlabel()`, `.set_ylabel()` и `.grid(True)` настраивают заголовки и сетку. `plt.tight_layout()` автоматически корректирует отступы.

Листинг:

```
fig, axs = plt.subplots(2, 2, figsize=(10, 8))
```

```
axs[0, 0].plot(x, x, 'b-')
axs[0, 0].set_title('y = x')
axs[0, 0].grid(True)
```

```
axs[0, 1].plot(x, x**2, 'r-')
axs[0, 1].set_title('y = x^2')
axs[0, 1].grid(True)
```

```
axs[1, 0].plot(x, np.sin(x), 'g-')
axs[1, 0].set_title('y = sin(x)')
axs[1, 0].grid(True)
```

```
axs[1, 1].plot(x, np.cos(x), 'm-')  
axs[1, 1].set_title('y = cos(x)')  
axs[1, 1].grid(True)
```

```
plt.tight_layout() plt.show()
```

Результат выполнения:

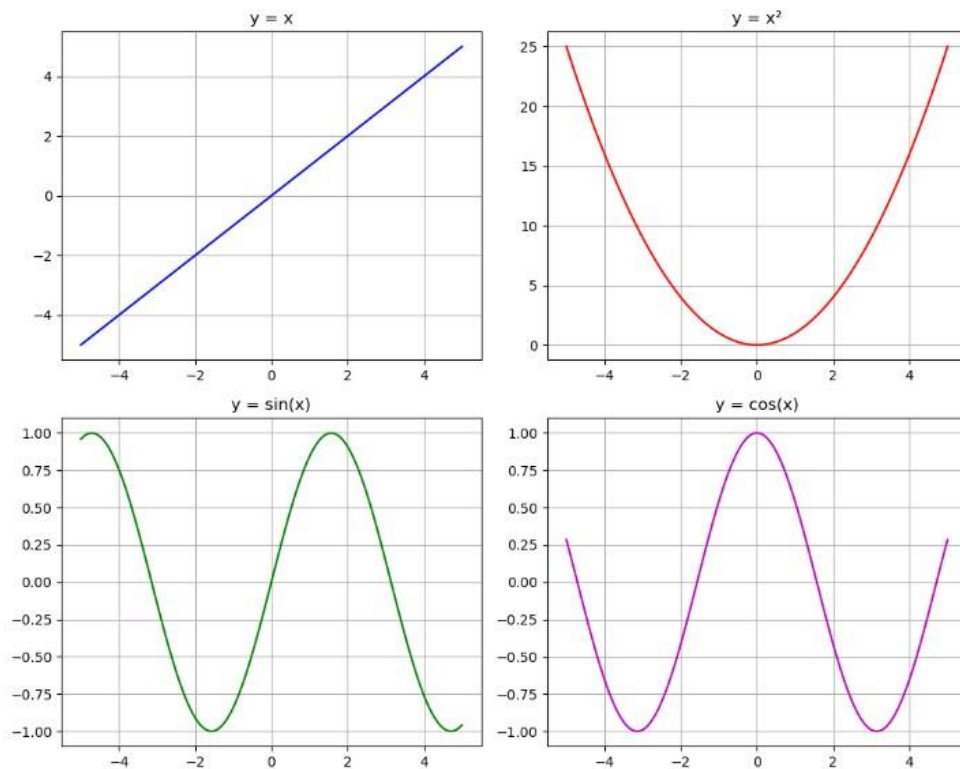


Рисунок 48 - Результат выполнения программы

Задание 9. Тепловая карта.

Создайте тепловую матрицу 10x10 с элементами от 0 до 1 и визуализируйте её, как тепловую карту с цветовой шкалой.

Решение: `np.random.seed(42)` фиксирует случайность. `np.random.rand(10, 10)` генерирует матрицу 10×10 со случайными числами от 0 до 1. `plt.imshow(data, cmap='viridis', interpolation='nearest')` отображает тепловую карту, где `cmap='viridis'` задаёт цветовую карту (фиолетовый – низкие значения,

жёлтый – высокие). `plt.colorbar()` добавляет цветовую шкалу. Оформляются заголовок и подписи осей.

Листинг:

```
np.random.seed(42) data =  
np.random.rand(10, 10)  
  
plt.imshow(data, cmap='viridis', interpolation='nearest')  
plt.colorbar(label=r'Значение')  
  
plt.title('Тепловая карта 10x10', fontsize=14)  
plt.xlabel('Столбцы', fontsize=12)  
plt.ylabel('Строки', fontsize=12) plt.show()
```

Результат выполнения:

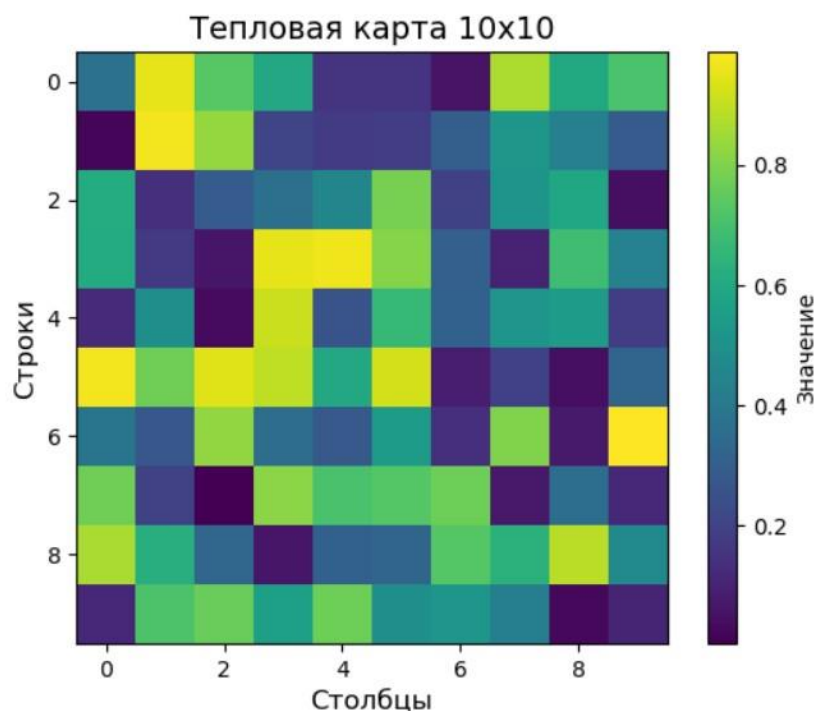


Рисунок 49 - Результат выполнения программы

Индивидуальные задания (вариант 6).

Задание 1. Среднесуточное потребление электроэнергии.

В доме фиксировали среднее потребление электроэнергии в течение суток:

1. Время суток (часы): [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24]
2. Потребление (кВт·ч): [0.8, 0.7, 0.6, 0.5, 0.9, 1.2, 1.5, 1.6, 1.7, 1.8, 1.6, 1.2, 1.0]

Постройте график с затемнением области под кривой.

Решение: создаются списки `time` (часы суток) и `using` (потребление в кВт·ч). Функция `plt.plot()` строит линейный график с синими кружками-маркерами ('b-o'), где линия соединяет точки наблюдений. `plt.fill_between(time, consumption, color='blue', alpha=0.3)` заливает область под кривой синим цветом с прозрачностью 30%, чтобы визуализировать суммарное потребление за сутки. `plt.grid(True)` добавляет пунктирную сетку.

Оформляются заголовок, подписи осей и легенда.

Листинг:

```
time = [0, 2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24]
using = [0.8, 0.7, 0.6, 0.5, 0.9, 1.2, 1.5, 1.6, 1.7, 1.8, 1.6, 1.2, 1.0]
```

```
plt.plot(time, using, 'b-o', linewidth=2, markersize=6, label='Потребление')
plt.fill_between(time, using, color='blue', alpha=0.3)
```

```
plt.title(r'Среднесуточное потребление электроэнергии', fontsize=14)
plt.xlabel(r'Время суток (часы)', fontsize=12)
plt.ylabel(r'Потребление (кВт·ч)', fontsize=12)
plt.grid(True)
plt.legend()
```

```
plt.show()
```

Результат выполнения:



Рисунок 50 - Результат выполнения программы

Задание 2. Вклад разных отраслей в экономику страны Данные

о доле различных отраслей в ВВП:

1. Отрасли: ['ИТ', 'Производство', 'Туризм', 'Финансы', 'Транспорт'] 2.

Доля в ВВП (%): [15, 25, 10, 30, 20]

Добавьте горизонтальную линию, показывающую среднее значение доли.

Решение: создаются списки `industries` (названия отраслей) и `share` (доля в ВВП в процентах). Функция `np.mean(share)` вычисляет среднее значение доли (20%). `plt.bar()` строит столбчатую диаграмму: каждому названию отрасли соответствует столбец высотой, равной доле. Параметр `color` задаёт индивидуальные цвета для каждого столбца, `edgecolor='black'` – чёрную обводку. `plt.axhline(y=mean_share, color='red', linestyle='--', linewidth=2)` добавляет горизонтальную пунктирную красную линию на уровне среднего значения. `plt.grid(True, axis='y')` включает сетку только по вертикальной оси. Цикл `for i, v in enumerate(share)` с `plt.text()` добавляет числовые подписи над

каждым столбцом, где `ha='center'` центрирует текст. Оформляются заголовок, подписи осей и легенда.

Листинг:

```
industries = ['ИТ', 'Производство', 'Туризм', 'Финансы', 'Транспорт'] share
= [15, 25, 10, 30, 20]

mean_share = np.mean(share) plt.bar(industries, share,
color=['#4CAF50', '#2196F3', '#FF9800',
'#9C27B0', '#F44336'], edgecolor='black', alpha=0.8) plt.axhline(y=mean_share,
color='red', linestyle='--', linewidth=2,
label=f'Среднее значение = {mean_share:.1f}%')

plt.title(r'Доля отраслей в ВВП страны', fontsize=14)
plt.xlabel(r'Отрасль', fontsize=12) plt.ylabel(r'Доля в
ВВП (%)', fontsize=12) plt.grid(True, axis='y')
plt.legend()

for i, v in enumerate(share):
    plt.text(i, v + 1, f'{v}%', ha='center', fontsize=10)

plt.show()
```

Результат выполнения:

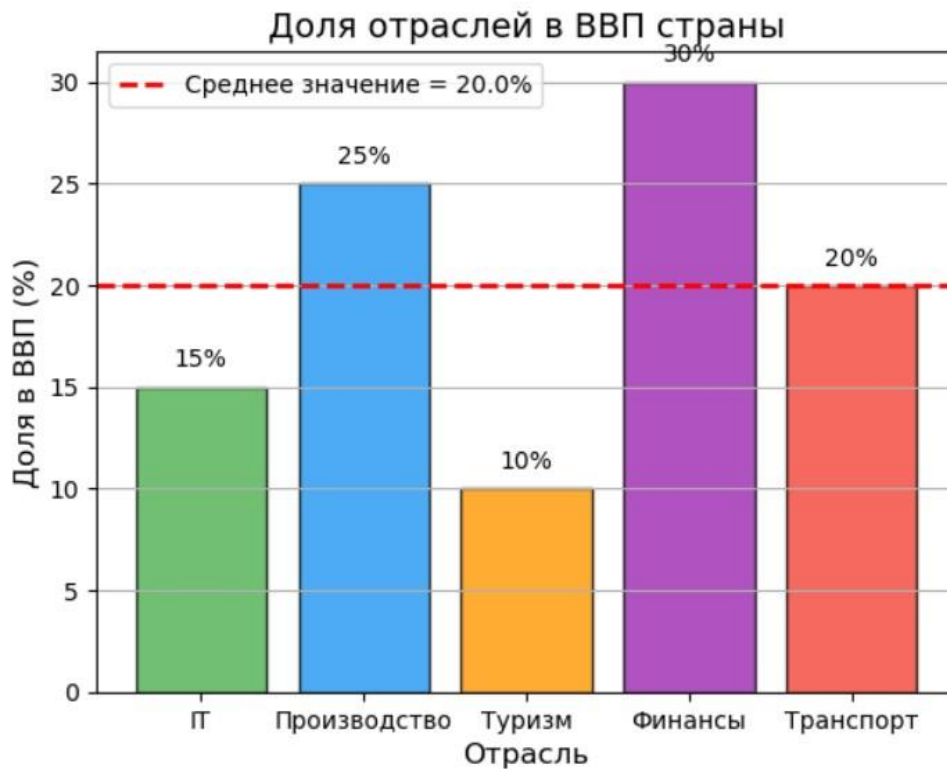


Рисунок 51 - Результат выполнения программы

Задание 3. Площадь под кубической параболой

Вычислите определенный интеграл для функции: $f(x) = x^3 - 2x^2 + x$ на отрезке $[-1, 2]$.

Требования:

1. Построить график подинтегральной функции.
2. Вычислить площадь под кривой на заданном отрезке как значение определенного интеграла.
3. Закрасить область под графиком, чтобы визуализировать интеграл.
4. Использовать функцию `fill_between()`.
5. Добавить подписи осей и заголовок.
6. Отобразить оси X и Y для лучшей визуализации.

Решение: из `scipy.integrate` импортируется функция `quad` для численного интегрирования. Определяется функция $f(x) = x^3 - 2x^2 + x$. `quad(f, a, b)` вычисляет определённый интеграл на отрезке $[-1, 2]$, возвращая значение площади и погрешность. Создаётся массив `x` с 500 точками для плавного графика. `plt.plot()` строит график функции синей линией. Для визуализации интеграла `plt.fill_between()` заливает зелёным с прозрачностью 30% область под кривой только на отрезке интегрирования. `plt.axhline(y=0)` и `plt.axvline(x=0)` отображают координатные оси чёрным цветом. `plt.axvline(x=a)` и `plt.axvline(x=b)` пунктирными красными линиями отмечают границы интегрирования. Оформляются заголовок, подписи осей, сетка и легенда с выведенным значением площади. Функция `print()` выводит численное значение интеграла в консоль.

Листинг:

```
from scipy.integrate import quad
```

```
def f(x):
```

```
    return x**3 - 2*x**2 + x
```

```
a, b = -1, 2
```

```
area, error = quad(f, a, b)
```

```
x = np.linspace(a - 0.5, b + 0.5, 500) y  
= f(x)
```

```
plt.figure(figsize=(10, 6)) plt.plot(x, y, 'b-', linewidth=2, label=f'$f(x) = x^3 -  
2x^2 + x$') x_fill = np.linspace(a, b, 500) y_fill = f(x_fill) plt.fill_between(x_fill,  
y_fill, alpha=0.3, color='green', label=f'Площадь = {area:.4f}')
```



```
plt.axhline(y=0, color='black', linestyle='-', linewidth=0.8) plt.axvline(x=0,
color='black', linestyle='-', linewidth=0.8)
```

```
plt.axvline(x=a, color='red', linestyle='--', alpha=0.7, label=f'x = {a}')
```

```
plt.axvline(x=b, color='red', linestyle='--', alpha=0.7, label=f'x = {b}')
```

```
plt.title(r'Площадь под кривой  $f(x) = x^3 - 2x^2 + x$  на отрезке  $[-1,$   
2]', fontsize=14) plt.xlabel(r'$x$',
```

```
    fontsize=12)
```

```
plt.ylabel(r'$y$',
```

```
    fontsize=12) plt.grid(True)
```

```
plt.legend(loc='upper left')
```

```
plt.show()
```

```
print(f'Определённый интеграл  $\int(\{a\} \text{ to } \{b\}) (x^3 - 2x^2 + x) dx =$   
{area:.4f}') print(f'Площадь под кривой = {area:.4f} квадратных  
единиц")
```

Результат выполнения:

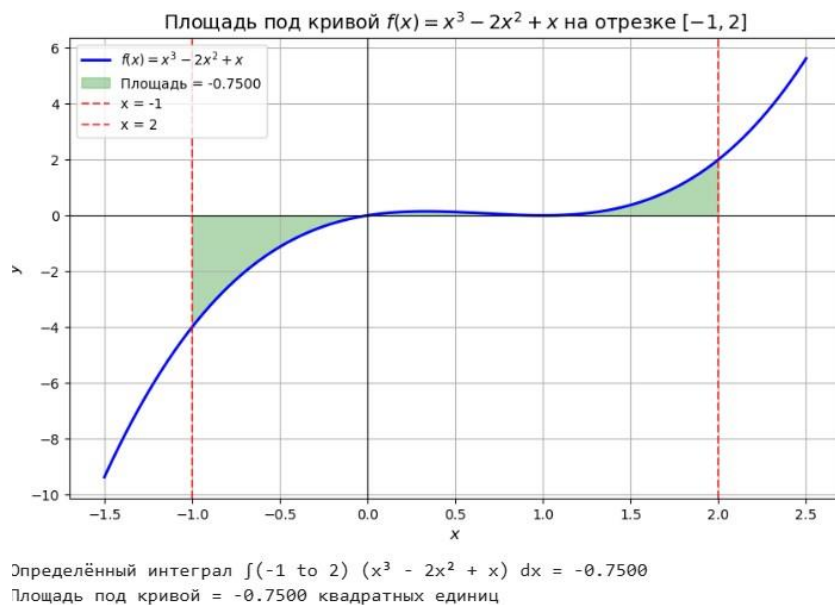


Рисунок 52 - Результат выполнения программы

Задание 4. Гиперболическая поверхность Постройте поверхность:

$$f(x, y) = \frac{1}{1 + x^2 + y^2}$$

на $x, y \in [-3, 3]$.

Требования:

1. Построить трехмерный график функции
2. Использовать библиотеку Matplotlib
3. Оформить график: добавить заголовок, подписи осей и цветовую карту
4. Использовать `plot_surface()` для плотной поверхности
5. Настроить цветовую карту через `cm`
6. Добавить подписи осей и заголовки

Решение: `np.linspace(-3, 3, 100)` создаёт 100 равномерных точек по каждому измерению. `np.meshgrid(x, y)` образует двумерные сетки X и Y для вычислений. Значения Z вычисляются по формуле $Z = 1 / (1 + X^2 + Y^2)$, что даёт поверхность в виде «холма» с максимумом 1 в центре ($x=0, y=0$) и убыванием к краям области. `fig = plt.figure(figsize=(12, 8))` задаёт размер полотна. `fig.add_subplot(111, projection='3d')` создаёт 3D-оси. `ax.plot_surface(X, Y, Z, cmap='viridis', edgecolor='none', alpha=0.9)` строит плотную залитую поверхность, где `edgecolor='none'` убирает чёрные грани для более гладкого вида, а цветовая карта `viridis` отображает высоту (фиолетовый – низкие значения, жёлтый/зелёный – высокие). `fig.colorbar(surf, shrink=0.5, aspect=10)` добавляет цветовую шкалу с уменьшенным размером.

Оформляются заголовок, подписи осей. `ax.grid(True)` включает сетку.

Листинг:

```
x = np.linspace(-3, 3, 100) y
= np.linspace(-3, 3, 100) X,
Y = np.meshgrid(x, y)
```

```
Z = 1 / (1 + X**2 + Y**2)
```

```
fig = plt.figure(figsize=(12, 8)) ax =
fig.add_subplot(111, projection='3d')
```

```
surf = ax.plot_surface(X, Y, Z, cmap='viridis', edgecolor='none', alpha=0.9)
ax.set_title(r'Гиперболическая поверхность  $f(x, y) = \frac{1}{1 + x^2 + y^2}$ ',
             fontsize=14) ax.set_xlabel(r'Ось
 $x$ ', fontsize=12) ax.set_ylabel(r'Ось
 $y$ ', fontsize=12) ax.set_zlabel(r'Ось
 $z$ ', fontsize=12)
```

```
fig.colorbar(surf, shrink=0.5, aspect=10, label=r'Значение  $f(x, y)$ ')
plt.show()
```

Результат выполнения:

Гиперболическая поверхность $f(x, y) = \frac{1}{1+x^2+y^2}$

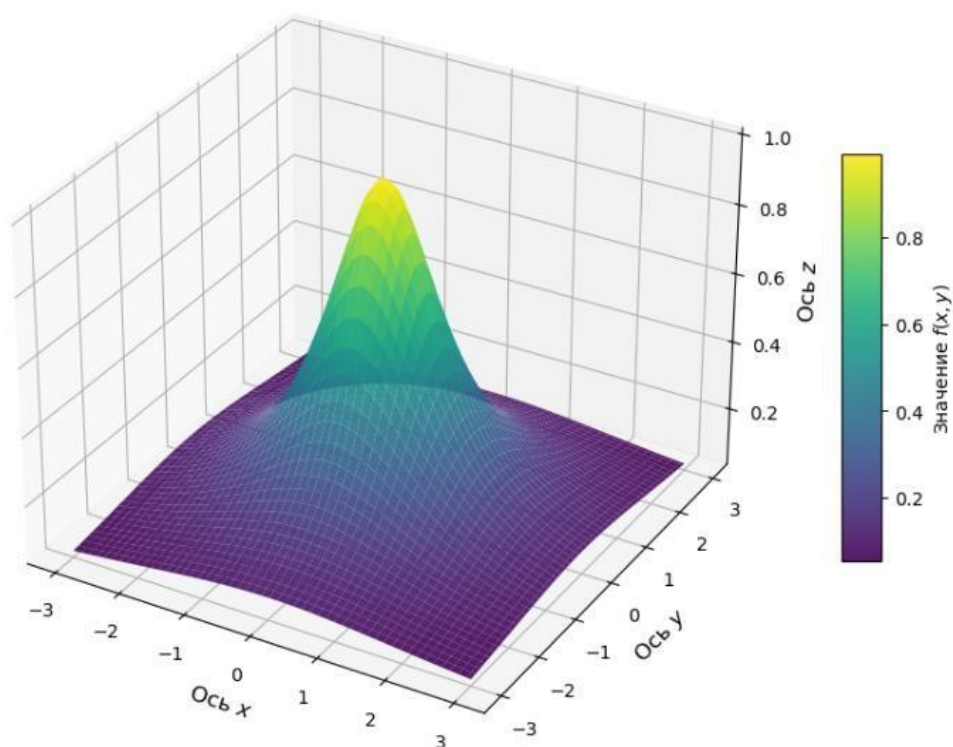


Рисунок 53 - Результат выполнения программы

Вывод: в ходе выполнения лабораторной работы были исследованы базовые возможности библиотеки Matplotlib языка программирования Python. В процессе работы освоены следующие основные приемы визуализации данных: построение линейных графиков функций с настройкой цвета, стиля и толщины линий; создание диаграмм рассеяния с зависимостью цвета и размера маркеров от значений координат; построение гистограмм распределения случайных величин с настройкой количества бинов; создание столбчатых и круговых диаграмм для визуализации категориальных данных; построение трехмерных графиков поверхностей и каркасных моделей; создание тепловых карт для визуализации матричных данных; оформление графиков с добавлением заголовков, подписей осей, легенды, сетки и цветовых шкал; работа с множественными подграфиками (subplots) для размещения нескольких графиков в одной фигуре; использование функций

`fill_between()` для заливки областей под кривыми и между кривыми, а также выборочной и двухцветной заливки. Таким образом, поставленная цель лабораторной работы достигнута: получены практические навыки работы с библиотекой Matplotlib, позволяющие решать широкий круг задач визуализации данных при анализе и обработке информации.

Контрольные вопросы:

1. Как осуществляется установка пакета matplotlib?

Установка выполняется через pip: `pip install matplotlib`. В среде Anaconda: `conda install matplotlib`.

2. Какая "магическая" команда должна присутствовать в ноутбуках Jupyter для корректного отображения графиков matplotlib?

`%matplotlib inline` – она встраивает графики прямо в ячейки ноутбука.

3. Как отобразить график с помощью функции plot? `plt.plot(x, y)` строит график, а `plt.show()` отображает его.

4. Как отобразить несколько графиков на одном поле?

Несколько раз вызвать `plt.plot()` для разных данных перед `plt.show()`.

5. Какой метод Вам известен для построения диаграмм категориальных данных?

`plt.bar()` для столбчатых диаграмм и `plt.pie()` для круговых.

6. Какие основные элементы графика Вам известны?

Заголовок, оси X и Y, подписи осей, сетка, легенда, маркеры, линии, деления (ticks).

7. Как осуществляется управление текстовыми надписями на графике? `plt.title()` – заголовок, `plt.xlabel()` / `plt.ylabel()` – подписи осей, `plt.text()` –

произвольный текст в координатах.

8. Как осуществляется управление легендой графика?

В `plt.plot()` добавить параметр `label='текст'`, затем вызвать `plt.legend()`.

9. Как задать цвет и стиль линий графика?

В `plt.plot()` указать формат: `'r--'` (красный пунктир) или параметры `color='red'`, `linestyle='--'`.

10. Как выполнить размещение графика в разных полях?

Использовать `plt.subplot(rows, cols, index)` или `plt.subplots()`.

11. Как выполнить построение линейного графика с помощью matplotlib?

`plt.plot(x, y)` – строит линию по точкам (x, y).

12. Как выполнить заливку области между графиком и осью? Между двумя графиками?

`plt.fill_between(x, y)` – заливка под кривой до оси. `plt.fill_between(x, y1, y2)` – заливка между двумя кривыми.

13. Как выполнить выборочную заливку, которая удовлетворяет некоторому условию?

Использовать параметр `where` в `fill_between()`: `plt.fill_between(x, y, where=(y > 0))`.

14. Как выполнить двухцветную заливку?

Два вызова `fill_between()` с разными условиями и цветами: для $y \geq 0$ и $y \leq 0$.

15. Как выполнить маркировку графиков?

Параметр `marker` в `plt.plot()`: `marker='o'` (кружки), `marker='^'` (треугольники) и т.д.

16. Как выполнить обрезку графиков?

`plt.xlim(min, max)` и `plt.ylim(min, max)` – задают диапазоны осей.

17. Как построить ступенчатый график? В чем особенность ступенчатого графика?

`plt.step(x, y)`. Особенность: значения постоянны на интервалах, меняются только в точках.

18. Как построить стековый график? В чем особенность стекового графика?

`plt.stackplot(x, y1, y2, ...)`. Особенность: области графиков суммируются вертикально (накопление).

19. Как построить stem-график? В чем особенность stem-графика?

`plt.stem(x, y)`. Особенность: вертикальные линии от оси до точек с маркерами на концах.

20. Как построить точечный график? В чем особенность точечного графика?

`plt.scatter(x, y)`. Особенность: отображает отдельные точки, можно задать цвет и размер каждой точки.

21. Как осуществляется построение столбчатых диаграмм с помощью matplotlib? `plt.bar(x, height)` – вертикальные столбцы, `plt.barh(x, width)` – горизонтальные.

22. Что такое групповая столбчатая диаграмма? Что такое столбчатая диаграмма с `errorbar` элементом?

Групповая – несколько столбцов для каждой категории со смещением. С `errorbar` – добавляет отрезки погрешностей (`yerr`).

23. Как выполнить построение круговой диаграммы средствами matplotlib?

`plt.pie(data, labels=labels)`.

24. Что такое цветовая карта? Как осуществляется работа с цветовыми картами в matplotlib?

Цветовая карта (`colormap`) – отображение чисел в цвета. Задаётся параметром `cmap` в `imshow`, `scatter`, `plot_surface`.

25. Как отобразить изображение средствами matplotlib?

`plt.imshow(image)`.

26. Как отобразить тепловую карту средствами matplotlib?

`plt.imshow(matrix, cmap='hot') + plt.colorbar()`.

27. Как выполнить построение линейного 3D-графика с помощью matplotlib?

`ax.plot3D(x, y, z)` или `ax.plot(x, y, z)` на 3D-осях.

28. Как выполнить построение точечного 3D-графика с помощью matplotlib?

`ax.scatter3D(x, y, z)` или `ax.scatter(x, y, z)` на 3D-осях.

29. Как выполнить построение каркасной поверхности с помощью matplotlib?

`ax.plot_wireframe(X, Y, Z)`.

30. Как выполнить построение трехмерной поверхности с помощью matplotlib?

`ax.plot_surface(X, Y, Z, cmap='viridis')`.